

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_a2b97cff-dcd\agent-aec779a62c7c024f1.jsonl`
- SHA-256: `38a59d01f39062b0a876df35ef4da8026c62754b0108bd762665f39c9f164343`
- Source modified: `2026-06-21T14:00:37+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_a2b97cff-dcd`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`
```

Transcript

1. user (2026-06-21T13:52:09.116Z)

Research running the SAME app "at cloud services" with minimal friction (the spec says local AND cloud). Cover: deploy-to-Cloud-Run (we already dispatch Cloud Run Jobs), Render/Railway/Fly.io one-click, a "Deploy" button, BYO-cloud via our existing provision-vm.sh/teardown-vm.sh, and how ONE artifact (same Docker image / same package) serves both local and cloud by config (deployment.profile). Critical: edge-auth before any public exposure (we have Cloudflare Access M9), per-video cost ledger (M8), and not leaking the user's data. Recommend how the downloadable app offers a "push my workload to cloud" mode. Use web search.

Return the RESEARCH schema. Context about our app:

PROJECT: a local, AI-powered badminton/racket-sports HIGHLIGHT INDEXER + rally detector.
Engine repo (PRIMARY): C:/Users/avidu/Projects/badminton-highlight-indexer (Python, FastAPI + SQLite + ffmpeg + GPU CV/AI; paid Gemini = cloud AI).
Sibling repos: C:/Users/avidu/Projects/khelsutra (the web/ React SPA + site/ marketing), C:/Users/avidu/Projects/rally-annotator (VLC plugin, MIT), C:/Users/avidu/Projects/sports-data-collector, C:/Users/avidu/Projects/sports-obsreport.

GOAL of the plan we are building: bake the engine into a DOWNLOADABLE, "batteries-included" Python APP that a user can download and run WITHOUT sourcing the repo (no git clone / build-from-source). It must spin up a local WEB SERVICE that runs (a) INFERENCE (video -> rallies -> highlight reel), (b) TRAINING (BYO model on their own labelled data), and (c) RALLY ANNOTATION if needed -- runnable BOTH locally and against CLOUD services. Preserve every already-completed end-to-end CUJ.

KEY FACTS the orchestrator already verified:

```
- pyproject.toml: hatchling build, namespace package (no backend/__init__.py),
packages=["backend","training"], console script: indexer = backend.main:main. requires-python
>=3.10.
- torch/torchvision are DELIBERATELY NOT in dependencies -- their CUDA/CPU wheels need --index-
url (PEP621 cannot express). Optional extras: gpu(empty, docs-only), auth(PyJWT),
reporting(git+ssh obsreport), storage-s3(boto3), storage-gcs(google-cloud-storage). google-genai
is a hard dep.
- Server today: python -m backend.main --server --port 8000 . Worker: python -m backend.worker
{infer|train} . CLI single-shot: python -m backend.main --video-path ... --segmenter
{gemini|yolo_hybrid|wasb_hybrid|motion}.
- THE HARD PACKAGING PROBLEM: native WASB shuttle model runs in a SEPARATE Python env (conda,
torch 1.11+cu113) while the main engine uses torch 2.6+cu124. ffmpeg/ffprobe are system deps.
Model weights (wasb_badminton_best.pth.tar) live in a GCS bucket.
- Decoupled-compute SEAM already built (M1-M8 merged, default-OFF): deployment.profile +
compute_target on AppConfig; backend/config/presets.py (PROFILE_PRESETS: truly-local / cloud-
serving / cpu-mock); backend/storage/* (local/gcs/s3/gdrive StorageBackend registry);
backend/compute/* (ComputeBackend registry, CloudRunCompute dispatches a Cloud Run Job);
backend/config/startup.py per-profile checks; backend/billing.py cost ledger.
- The web SPA lives in a DIFFERENT repo (khelsutra/web, Vite/React/TS/Tailwind, i18n en+kn+hi).
```

The engine has a likely-DEAD backend/api/static. Bundling the UI is a cross-repo concern.

- Licensing guardrail (NON-NEGOTIABLE): every dependency -- code, data, AND weights -- must be MIT/Apache-2.0/BSD only. Paid LLMs = inference/serving only, never training data. Any NEW packaging dependency must also be license-clean.
- A prior docs/DESKTOP_APP_PLAN.md exists but was SUPERSEDED by owner decision D10: deliver a "self-hosted webserver + UI the user spins up" (NOT a desktop app).

khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md is the live appliance design.

Always ground claims in file:line or a doc path. Read the actual files; do not assume.

2. assistant (2026-06-21T13:52:12.325Z)

I'll research this thoroughly. Let me start by reading the relevant files to ground my claims in the actual codebase.

3. user (2026-06-21T13:52:13.964Z)

```
"""Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING M1).
```

A *deployment profile* is a single switch that applies a coherent BUNDLE of existing config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one knob at a time (the design in ``docs/COMPUTE\_DECOUPLED\_SERVING/README.md`` §4.3).

Precedence (lowest ? highest), realised by the loader::

```
built-in model defaults < profile preset < config.json < env < worker --set
```

This module is *pure data + pure functions* ? no IO, and (critically) *no torch import at module load*. The loader applies a preset ONLY when a profile is EXPLICITLY selected (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT\_PROFILE`` env var); with no profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` == today). Auto-detect only ever *suggests* a profile ? it never auto-applies one (D15: cloud spend is never entered silently).

```
"""
```

```
from __future__ import annotations
```

```
import copy
import os
from typing import Any, Mapping, Optional
```

Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal`` these and ``tests/test\_config\_deployment.py`` pins parity so the two can never drift.

```
PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
```

Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to when config.json explicitly overrides compute_target into a state inconsistent with the

```
COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
    "truly-local": "local_gpu",
    "cloud-serving": "cloud_run",
    "cpu-mock": "cpu_mock",
}
```

A preset is a partial, nested override dict over the model defaults. It touches ONLY knobs that EXIST today ? M1 is pure/additive: the new ``deployment.{profile,compute\_target,indexing.{detector\_impl,skip\_ai\_handoff}}`` and ``storage.backend``. Later milestones these bundles ? ``input\_backend`` (M2) and the configurable output/workspace base (Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.

```

PROFILE_PRESETS: dict[str, dict[str, Any]] = {
    "truly-local": {
        "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
        # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU box).
        # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
        "indexing": {"skip_ai_handoff": True},
        "storage": {"backend": "local"},
    },
    "cloud-serving": {
        "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
        # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor (D11).
        "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
        "storage": {"backend": "gcs"},
    },
    "cpu-mock": {
        "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
        # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression profile.
        "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
        "storage": {"backend": "local"},
    },
}

```

```

def is_known_profile(profile: str) -> bool:
    """True for a non-empty, recognised profile name."""
    return bool(profile) and profile in PROFILE_PRESETS

```

```

def preset_for(profile: str) -> dict[str, Any]:
    """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate the
    module-level table), or ``{}`` for ``""`` / an unknown profile."""
    return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))

```

```

def _truthy(val: Optional[str]) -> bool:
    """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
    if val is None:
        return False
    return val.strip().lower() in {"1", "true", "yes", "on"}

```

```

def _cuda_available() -> bool:
    """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here and
    ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no GPU'."""
    try:
        # Intentionally lazy + local: keeps the config-load path torch-free (torch is heavy
        # and optional). Only reached when a caller opts into probe_cuda.
        import torch

        return bool(torch.cuda.is_available())
    except Exception:
        return False

```

```

def probe_cuda_available() -> bool:
    """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).

```

Exposed for the per-profile startup checks (``backend/config/startup.py``) so a caller that *wants* the GPU heads-up can pass the result in without the startup module importing torch itself. Best-effort by design: if torch lives only in a detector subprocess env (native WASB), this returns ``False`` in the main process ? which is why the startup checks treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a hard error that could false-fail."""

```

return _cuda_available()

```

```

def suggest_profile(
    env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
) -> str:
    """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.

    Signals, in priority order:

        - ``K_SERVICE`` set (Cloud Run)           ? ``"cloud-serving"``
        - ``CI`` truthy                             ? ``"cpu-mock"``
        - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
        - otherwise                                 ? ``""`` (no strong suggestion)

    ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and never
    imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe passes
    ``probe_cuda=True``. The result is advisory only: the caller decides whether to act on it.
    """
    e = os.environ if env is None else env
    if e.get("K_SERVICE"):
        return "cloud-serving"
    if _truthy(e.get("CI")):
        return "cpu-mock"
    if probe_cuda and _cuda_available():
        return "truly-local"
    return ""

```

4. user (2026-06-21T13:52:14.413Z)

```

backend\compute\__init__.py
backend\compute\cloud_run.py
backend\compute\base.py

```

5. user (2026-06-21T13:52:14.945Z)

```

backend\storage\gcs.py
backend\storage\local.py
backend\storage\s3.py
backend\storage\base.py
backend\storage\gdrive.py
backend\storage\ref.py
backend\storage\__init__.py

```

6. user (2026-06-21T13:52:14.959Z)

```

deploy\digitalocean\bootstrap.sh
deploy\digitalocean\mount_scratch.sh
deploy\digitalocean\gpu_setup.sh
deploy\gcp\teardown.sh
deploy\digitalocean\README.md
deploy\gcp\README.md
deploy\cloudrun\Dockerfile
deploy\cloudrun\README.md
deploy\gcp\provision.sh
deploy\gcp\ops-agent-startup.sh
deploy\gcp\create_gpu_vm.sh

```

7. assistant (2026-06-21T13:52:17.841Z)

Let me read the core decoupled-compute files and the deploy infrastructure.

8. user (2026-06-21T13:52:18.934Z)

```

1      """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2      for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).

```

```

3
4     Flow (the cloud-serving compute path):
5     1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6         --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7     2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8         completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9         it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10        cold-start is absorbed by the async poll.
11    3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12        ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
14    The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
15    so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17    The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
18    with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
19    owner-verified on the M7 real run.
20    ""
21
22    from __future__ import annotations
23
24    import json
25    import logging
26    import os
27    import tempfile
28    import uuid
29    from abc import ABC, abstractmethod
30    from typing import Any, List, Optional
31
32    from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
33    from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
34    from backend.storage import fetch, parse_uri
35    from backend.storage import get_storage
36
37    LOG = logging.getLogger("compute.cloud_run")
38
39    # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
40    # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
41    _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
42        "deployment.compute_target=local_gpu",
43        "indexing.detector_impl=native",
44    )
45
46
47    class CloudRunJobClient(ABC):
48        """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
49
50        Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
51        overrides the job's container ``args`` for this execution and starts it."""
52
53        @abstractmethod

```

```

54     def run_job(self, job_name: str, argv: List[str], *, region: str,
55                 project: Optional[str] = None) -> str:
56         """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
57         execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
58
59
60     class GoogleCloudRunJobClient(CloudRunJobClient):
61         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
62         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
63
64     def run_job(self, job_name: str, argv: List[str], *, region: str,
65                 project: Optional[str] = None) -> str:
66         try:
67             from google.cloud import run_v2 # type: ignore[attr-defined]
68             except Exception as exc: # pragma: no cover - real SDK not present in CI
69                 raise RuntimeError(
70                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
71                     "control plane (it is intentionally NOT a CI/test dependency).")
72             ) from exc
73             client = run_v2.JobsClient() # pragma: no cover - exercised only on the real M7
run
74             name = client.job_path(project, region, job_name)
75             overrides = run_v2.RunJobRequest.Overrides(
76                 container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
77             )
78             op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
79             return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"
80
81
82     class CloudRunCompute(ComputeBackend):
83         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
84
85     def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
86                 project: Optional[str] = None, storage_config: Optional[dict] = None,
87                 policy: ExecutionPolicy = DEFAULT_POLICY,
88                 token: Optional[str] = None,
89                 container_overrides: Optional[tuple[str, ...]] = None) -> None:
90         self._client = run_client
91         self._job_name = job_name
92         self._region = region
93         self._project = project
94         self._storage_config = storage_config or {}
95         self._policy = policy
96         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
97         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
98         self._token = token
99         self._container_overrides = (
100             _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
101         )
102
103     def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
104         out_root = spec.out_uri.rstrip("/")
105         token = self._token or uuid.uuid4().hex
106         done_uri = f"{out_root}/_dispatch/{token}.done"

```

```

107         argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
108                             "--done-uri", done_uri]
109         if spec.segmenter:
110             argv += ["--segmenter", spec.segmenter]
111         for kv in (*spec.config_overrides, *self._container_overrides):
112             argv += ["--set", kv]
113
114         execution = self._client.run_job(self._job_name, argv, region=self._region,
115                                         project=self._project)
116         LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
117                 self._job_name, execution, done_uri)
118
119         marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
120                             label="cloud-run-infer", logger=LOG)
121         video_id = str(marker.get("video_id", ""))
122         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
123         # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
124         rc = int(marker.get("returncode", 1))
125         if not video_id:
126             LOG.warning("cloud-run: job=%s completion marker present but
empty/unreadable (no "
127                         "video_id) ? treating as failure", self._job_name)
128             rc = rc or 1
129         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
130         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
131         LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
132         return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
133
134         # --- storage-backed completion signal (bucket-poll) ---
135         def _read_marker(self, done_uri: str) -> Optional[dict]:
136             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
137             ref = parse_uri(done_uri)
138             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
139                 return None
140             return self._read_json(done_uri) or {} # empty dict still terminates the poll
(present == done)
141
142         def _read_json(self, uri: str) -> dict:
143             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
144             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
145             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
146             try:
147                 ref = parse_uri(uri)
148                 dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or
"obj.json")
149                 local = fetch(uri, dst, config=self._storage_config)
150                 with open(local, "r", encoding="utf-8") as f:
151                     data: Any = json.load(f)
152                 return data if isinstance(data, dict) else {}
153             except Exception:
154                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
155                 return {}
156

```

```

1      # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2      #
3      # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4      # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5      # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6      # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7      #
8      # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9      # native GPU box, deploy/digitalocean/gpu_setup.sh):
10     # * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11     # * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
12     # (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
13     #
14     # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
15     # commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
16     # mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
17
18     FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
19
20     ENV DEBIAN_FRONTEND=noninteractive \
21         PYTHONUNBUFFERED=1 \
22         PIP_NO_CACHE_DIR=1 \
23         CONDA_DIR=/opt/conda \
24         WASB_REPO_DIR=/opt/models/WASB-SBDT
25
26     # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
27     RUN apt-get update && apt-get install -y --no-install-recommends \
28         ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
29         python3 python3-pip python3-venv \
30         && rm -rf /var/lib/apt/lists/*
31
32     # --- WASB detector env (mirrors deploy/digitalocean/gpu_setup.sh ? the verified
torch-1.11 stack) ---
33     RUN curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh \
34         && bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh \
35         && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/main || true \
36         && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/r || true \
37         && "$CONDA_DIR/bin/conda" create -y -q -n wasb python=3.8 \
38         && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
39         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
40             torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
41         && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
42         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
43             hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib "numpy<1.24"
44
45     # --- Main application env ---
46     WORKDIR /app
47     COPY pyproject.toml README.md ./

```

```

48     COPY backend ./backend
49     COPY training ./training
50     RUN python3 -m pip install --upgrade pip && python3 -m pip install .
51
52     # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
53     # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
54     ENV WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
55         WASB_REPO_DIR=/opt/models/WASB-SBDT \
56         WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
57         RALLY_DETECTOR_IMPL=native
58
59     # The dispatcher (CloudRunCompute) supplies the per-execution args:
60     # infer --video-uri gs:/// --out-uri gs:/// --done-uri gs:/// --segmenter wasb_hybrid
--set ?
61     ENTRYPOINT ["python3", "-m", "backend.worker"]
62

```

10. user (2026-06-21T13:52:19.665Z)

```

1     # Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)
2
3     The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
4     dispatches a Cloud Run **Job** execution running this image's worker on an **L4**, then
**bucket-polls**
5     a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.
6
7     > **Status:** the dispatch shim + image are **landed + mock-tested** (M6). The **real
build + push +
8     > run on an L4** is the **M7** owner step, within the **$100/?10k** budget (D17) ?
confirm cmd + hourly
9     > cost first, **delete every resource after** ([[gcp-vm-always-delete]]). CI does
**not** build this image.
10
11     ## 0. Prerequisites (owner)
12     - A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
enabled.
13     - GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else
Singapore ? see
14     `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
15     - The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb_badminton_best.pth.tar`.
16
17     ## 1. Build + push the image (Artifact Registry)
18     ```bash
19     PROJECT=<gcp-project>; REGION=asia-south1; REPO=rally; IMG=rally-worker
20     gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
21     gcloud builds submit --tag $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest -f
deploy/cloudrun/Dockerfile .
22     ```
23     *(A CUDA + conda image is large; the first build is slow. Budget-watch per D17.)*
24
25     ## 2. Create the Cloud Run **Job** (GPU, scale-to-zero)
26     ```bash
27     gcloud run jobs create rally-worker \
28         --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
29         --region $REGION --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi \
30         --max-retries 0 --task-timeout 3600 \
31         --set-env-vars WASB_WEIGHTS=/staged/wasb_badminton_best.pth.tar
32     # (stage the weights into the container at start, or fetch from gs:// in an init step ?
WASB-C3.)
33     ```

```

```

34
35     ## 3. Wire the dispatcher (control plane)
36     The control plane runs with the cloud-serving profile (DEPLOYMENT_PROFILE=cloud-serving` ?
37     compute_target=cloud_run`). get_compute_backend(config)` then builds CloudRunCompute`
38     from env:
39     ```bash
40     export DEPLOYMENT_PROFILE=cloud-serving
41     export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION
42     GOOGLE_CLOUD_PROJECT=$PROJECT
43     # A worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
44     instead of
45     # running in-process; the container runs the SAME worker INLINE (compute_target forced
46     local_gpu`).
47     python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
48     gs://<bucket> --segmenter wasb_hybrid
49     ```
50
51     ## 4. M7 acceptance (the owner runlog)
52     Capture a DEPLOY_REPORT_cloud-serving_<date>.md` under
53     docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
54     commands,
55     rally counts, gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test
56     (ffmpeg -f null -`
57     exit 0 + a 5-frame contact sheet), gotchas, and the teardown audit.
58
59     ## Gotchas
60     - The dispatched container must run inline ? CloudRunCompute` forces
61     compute_target=local_gpu`
62     + indexing.detector_impl=native` in the per-execution args so the Job never re-
63     dispatches itself.
64     - Completion is a bucket marker (<out_uri>/_dispatch/<token>.done`), not the Cloud
65     Run API status
66     ? so it reuses the storage layer + execution_policy.poll_until` and survives a
67     control-plane restart.
68     - A failed Job currently writes no marker ? the dispatcher's bounded poll times
69     out
70     (ExecutionPolicy.max_wait_sec`) and _dispatch_remote` returns a clean rc 4 (not
71     a traceback).
72     A failure-marker (carrying the real non-zero rc, so the dispatcher distinguishes
73     failed from
74     hung/cold-start-slow) is a follow-up.
75     - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
76     libx264.
77
78

```

11. assistant (2026-06-21T13:52:22.512Z)

Now let me read the auth (M9 / Cloudflare Access), billing (M8), and the GCP provisioning scripts.

12. user (2026-06-21T13:52:23.482Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer.

13. user (2026-06-21T13:52:23.961Z)

```

1     """Per-job cost ledger ? pure cost math (COMPUTE_DECOUPLED_SERVING M8, decision D8).
2
3     ``cost_usd(job) = ?_resource usage(job, resource) × rate(resource, pricebook_version)``
4     (the formula from the archived 05-COST-ATTRIBUTION design). Cost is computed + stored in
5     USD
6
7     (matches GCP billing = one source of truth) and displayed in INR at a configurable

```

```

FX rate
6      (the Bangalore market). The pricebook is a versioned DB table
(``backend/infrastructure/database.py``);
7      this module is the pure arithmetic over a resolved ``{resource: unit_price_usd}`` rates
dict, so it
8      is unit-tested with zero IO and the DB layer owns the version/gpu_type lookup.
9
10     **Default-OFF:** with the seeded ``placeholder-v0`` pricebook (every rate ``0.0``) every
cost is
11     ``0.0`` ? nothing user-facing changes until the owner inserts a real, calibrated
pricebook version.
12     """
13
14     from __future__ import annotations
15
16     from typing import Dict, Mapping, Tuple
17
18     # Metered resource keys ? shared by the usage dict, the pricebook rows, and the
breakdown.
19     GPU_SECONDS = "gpu_seconds"      # GPU-active seconds (the big one) ? rate is per-second,
per gpu_type
20     WALL_SECONDS = "wall_seconds"    # total wall time (separate from GPU so idle/sync isn't
billed as GPU)
21     EGRESS_GB = "egress_gb"          # reel/byte egress in GB
22     GEMINI_USD = "gemini_usd"        # provider-reported cost, ALREADY in USD (a passthrough,
not rate-x)
23
24     # Resources whose cost = amount × unit_price (GEMINI_USD is excluded ? it is already a
USD amount).
25     _RATE_MULTIPLIED = (GPU_SECONDS, WALL_SECONDS, EGRESS_GB)
26
27
28     def compute_cost_usd(usage: Mapping[str, float],
29                          rates: Mapping[str, float]) -> Tuple[float, Dict[str, float]]:
30         """Return ``(cost_usd, breakdown)`` for one job.
31
32         ``usage`` maps a resource key to its measured amount (seconds / GB); ``rates`` maps
a resource
33         key to its USD unit price (already resolved for this pricebook version + gpu_type by
the DB
34         layer). ``GEMINI_USD`` in ``usage`` is added verbatim (it is already a USD cost, not
multiplied).
35         Missing amounts/rates default to ``0.0`` ? with the placeholder pricebook the total
is ``0.0``.
36         """
37         breakdown: Dict[str, float] = {}
38         total = 0.0
39         for res in _RATE_MULTIPLIED:
40             amount = float(usage.get(res, 0.0) or 0.0)
41             rate = float(rates.get(res, 0.0) or 0.0)
42             cost = round(amount * rate, 6)      # round the LINE, then sum rounded lines so
the
43             if amount or rate:                  # breakdown reconciles to the total exactly
(no drift),
44                 breakdown[res] = cost          # and record the line even at $0 for
transparency
45             total += cost
46         gemini = round(float(usage.get(GEMINI_USD, 0.0) or 0.0), 6)
47         if gemini:
48             breakdown[GEMINI_USD] = gemini
49         total += gemini
50         return round(total, 6), breakdown
51
52
53     def apply_margin(cost_usd: float, margin: float) -> float:

```

```

54         """Resale price = ``cost_usd * (1 + margin)``. ``margin`` is clamped ? 0 (never
below cost)."""
55         return round(cost_usd * (1.0 + max(0.0, float(margin))), 6)
56
57
58     def to_inr(cost_usd: float, fx_inr_per_usd: float) -> float:
59         """Display conversion: USD ? INR at the configured FX rate (2 dp, the display
currency)."""
60         return round(float(cost_usd) * float(fx_inr_per_usd), 2)
61

```

14. user (2026-06-21T13:52:24.325Z)

```

1      #!/usr/bin/env bash
2      # Provision the DECOUPLED_COMPUTE GCP storage + IAM (asia-south1 / Mumbai). Idempotent:
safe to
3      # re-run; already-existing resources are left as-is. The GPU VM is NOT created here (it
is gated on
4      # the GPUS_ALL_REGIONS quota ? see deploy/gcp/README.md §2-3); create it with the §3
command once
5      # the quota is granted. This documents EXACTLY what was run on 2026-06-20 (D0->GCP
pivot, ADR-010).
6      #
7      # Prereq: `gcloud auth login`; Service Usage API + a linked billing account enabled in
the console
8      # (README §1). Run: bash deploy/gcp/provision.sh
9      set -uo pipefail
10
11     PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
12     REGION="${RALLY_GCP_REGION:-asia-south1}"
13     BUCKET="${RALLY_GCS_BUCKET:-khelesutra-rally-corpus}"
14     BILLING_ACCT="${RALLY_BILLING_ACCT:-01420D-419EE0-53D83C}"
15     SA_NAME="rally-worker"
16     SA_EMAIL="${SA_NAME}@${PROJECT}.iam.gserviceaccount.com"
17     SA_KEY="${RALLY_SA_KEY:-$HOME/.gcp/rally-worker-sa.json}"
18     export CLOUDSDK_CORE_DISABLE_PROMPTS=1
19
20     echo "== project=$PROJECT region=$REGION bucket=gs://$BUCKET =="
21
22     echo "== enable APIs =="
23     # logging + monitoring are REQUIRED for the Ops Agent ? without them the agent installs
but its
24     # metric/log exports get PermissionDenied/SERVICE_DISABLED (the empty Observability
graphs cause).
25     gcloud services enable compute.googleapis.com cloudbilling.googleapis.com \
26         billingbudgets.googleapis.com iam.googleapis.com cloudresourcemanager.googleapis.com \
27         logging.googleapis.com monitoring.googleapis.com \
28         --project "$PROJECT"
29
30     echo "== GCS bucket (asia-south1, uniform bucket-level access) =="
31     gcloud storage buckets create "gs://$BUCKET" --location="$REGION" \
32         --uniform-bucket-level-access --project "$PROJECT" || echo " (bucket exists ? ok)"
33
34     echo "== service account + Storage Object Admin + key =="
35     gcloud iam service-accounts create "$SA_NAME" --display-name="Rally worker (storage)" \
36         --project "$PROJECT" || echo " (SA exists ? ok)"
37     gcloud projects add-iam-policy-binding "$PROJECT" \
38         --member="serviceAccount:$SA_EMAIL" --role="roles/storage.objectAdmin"
--condition=None \
39         --format="value(etag)" >/dev/null
40     # Ops Agent (GPU/NVML metrics + logs ? Cloud Monitoring/Logging) needs these on the VM's
SA, else
41     # its exports are PermissionDenied. Granted here so EVERY VM (created via
create_gpu_vm.sh) reports.
42     echo "== Ops Agent SA roles (monitoring.metricWriter + logging.logWriter) =="

```

```

43     for ROLE in roles/monitoring.metricWriter roles/logging.logWriter; do
44         gcloud projects add-iam-policy-binding "$PROJECT" \
45             --member="serviceAccount:$SA_EMAIL" --role="$ROLE" --condition=None \
46             --format="value(etag)" >/dev/null
47     done
48     if [ ! -f "$SA_KEY" ]; then
49         mkdir -p "$(dirname "$SA_KEY")"
50         gcloud iam service-accounts keys create "$SA_KEY" --iam-account="$SA_EMAIL"
51         echo "  key -> $SA_KEY  (NEVER commit this; export
GOOGLE_APPLICATION_CREDENTIALS=$SA_KEY)"
52     else
53         echo "  key already at $SA_KEY ? leaving it (delete to rotate)"
54     fi
55
56     echo "== \$100 budget alert (50/90/100%) =="
57     gcloud billing budgets create --billing-account="$BILLING_ACCT" \
58         --display-name="rally-100usd-guard" --budget-amount=100 \
59         --filter-projects="projects/$PROJECT" \
60         --threshold-rule=percent=0.5 --threshold-rule=percent=0.9 --threshold-rule=percent=1.0
\
61     || echo "  (budget may already exist ? check: gcloud billing budgets list --billing-
account=$BILLING_ACCT)"
62
63     echo "== GPU quota (the VM gate) =="
64     echo "  regional $REGION:"
65     gcloud compute regions describe "$REGION" --project "$PROJECT" \
66         --flatten="quotas[]" --format="table(quotas.metric,quotas.limit)" | grep -E
"NVIDIA_(T4|L4)_GPUS " || true
67     echo "  global:"
68     gcloud compute project-info describe --project "$PROJECT" \
69         --flatten="quotas[]" --format="table(quotas.metric,quotas.limit)" | grep
GPUS_ALL_REGIONS || true
70     echo ""
71     echo "If GPUS_ALL_REGIONS == 0, request an increase (README §2), then create the VM
(README §3)."
```

15. assistant (2026-06-21T13:52:28.205Z)

The auth.py file is untracked (in git status `?? backend/api/auth.py`). Let me find it and the M9 auth docs, plus the appliance architecture doc and provision-vm scripts.

16. user (2026-06-21T13:52:31.374Z)

```

total 33
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:20 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:20 ..
-rw-r--r-- 1 avidu 197609   59 Jun 18 15:52 __init__.py
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:19 __pycache__
-rw-r--r-- 1 avidu 197609 9728 Jun 21 16:45 job_worker.py
drwxr-xr-x 1 avidu 197609    0 Jun 11 20:44 static
-rw-r--r-- 1 avidu 197609 6882 Jun 18 15:52 uploads.py
---PROVISION-VM---
README.md
create_gpu_vm.sh
ops-agent-startup.sh
provision.sh
teardown.sh
---DOCS COMPUTE_DECOUPLED---
README.md
TESTING_STRATEGY.md
architecture.svg
runlogs
```

17. user (2026-06-21T13:52:31.398Z)

No files found

18. user (2026-06-21T13:52:32.167Z)

No files found

19. user (2026-06-21T13:52:32.421Z)

C:/Users/avidu/Projects/khelsutra/docs\LOCAL_APPLIANCE_ARCHITECTURE.md

20. assistant (2026-06-21T13:52:35.723Z)

The git status said `?? backend/api/auth.py` but it doesn't exist now (this is a snapshot in time). Let me search the codebase for the M9 auth implementation and Cloudflare Access, plus read the appliance architecture doc.

21. user (2026-06-21T13:52:36.807Z)

Found 19 files
pyproject.toml
docs\COMPUTE_DECOUPLED_SERVING\README.md
backend\config\models.py
docs\DOC_STATUS.md
docs\NEXT_STEPS.md
docs\archives\decisions\DECISIONS.md
docs\UI_PROPOSAL.md
docs\UI_ALPHA_EXECUTION_PLAN.md
docs\SETUP_NEW_MACHINE.md
docs\STORAGE_SHARING_MODEL.md
docs\HOSTING_PLAN.md
docs\COMPUTE_DECOUPLED_SERVING\TESTING_STRATEGY.md
docs\archives\past_projects\DECOUPLED_COMPUTE\03-PLATFORM-OPTIONS.md
artifacts\appliance_design.json
artifacts\so.json
artifacts\so.py
backend\storage\s3.py
artifacts\landing\index.html
artifacts\site_deploy\index.html

22. user (2026-06-21T13:52:37.828Z)

Found 1 file
docs\archives\past_projects\DECOUPLED_COMPUTE\01-EXISTING-PLANS-AND-GAP.md

23. user (2026-06-21T13:52:38.697Z)

```
1      # Single-box appliance ? UI-powered rally indexer + reel generator (KhelSutra Studio)
2
3      > **Status:** `DRAFT` ? owner-gated (auth choice, retention, cost ceiling, GPU host, P5
bridge) . **Owner:** Avi . **Created:** 2026-06-20 . **Last updated:** 2026-06-20
4      > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5      > **Tracking anchors:** $9 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
6      > **Relation:** peer-of `PER_RALLY_SELECTION.md` / `REQUEST_ACCESS_FORM.md` in this
repo; **consumes** the engine's decoupled `StorageBackend` + `python -m backend.worker` seams
and the FastAPI loop; **extends** the engine's `HOSTING_PLAN.md` gating + `UI_PROPOSAL.md` alpha
screens. Engine-side changes here are **proposals to coordinate** with the active
decoupled/worker sessions ? not written unilaterally.
7      > **Honesty note:** every engine-fact claim is tagged `[verified]` (read against engine
source this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not
legal/financial advice.
```

```

8
9    ---
10
11    ## 0. TL;DR
12    **KhelSutra Studio** turns the engine from a CLI into a **single-box, UI-powered rally
indexer + reel generator**: the khelsutra `web/` React SPA, served behind an **edge auth gate**,
drives the **existing** engine FastAPI loop (ingest ? index ? pick rallies ? reel ? download) on
**one machine**, with `StorageBackend=local` for the MVP. It is the **single-box deployment
profile of the just-shipped decoupled architecture** ? storage=local + compute=local is the
degenerate collapse of the same storage/worker split.
13
14    The **single most important caveat**: the engine has **zero app-layer auth and CORS
`***` `[verified backend/main.py:42]`, so safety is structural **only** because the engine stays
bound to `127.0.0.1:8000` `[verified backend/main.py:638]`, a firewall blocks inbound `8000`,
and a reverse proxy/tunnel is the sole authenticated ingress ? **the gate, not the app, is the
tenancy boundary.**
15
16    Two honest hardware truths: (1) the current droplet is **CPU-only** ? real detection
there is **Gemini (cloud)** or **stub (CPU mock)**; **real local-GPU detection needs a GPU
host** (native WASB, in-flight). (2) The owner priority is to **stand up a GPU host ASAP** ?
provider-agnostic (DO GPU droplet / GCE GPU VM / Cloud Run+GPU) ? but that runs **in parallel**
with the UI MVP, which ships on Gemini today.
17
18    ## 1. Problem & goal
19    The decoupled engine just shipped a `StorageBackend` ABC + a `python -m backend.worker
{infer|train}` dispatcher `[verified backend/storage/, backend/worker/__main__.py]`, and the
FastAPI loop already covers `process ? jobs ? query ? compile ? highlights` `[verified
backend/main.py]`. What's missing is a **console** that lets a non-CLI operator run the whole
loop on a single box, **honestly, behind a gate**.
20
21    **Headline CUJ (owner's words)**: a **hosted website** where I pick an input ? **a
Google Drive file, a bucket file (`s3:///gs://`), or a local upload** ? it lands in the
**bucket (DO Spaces / GCS)** or locally ? I **generate highlights** on a **GPU-enabled host**
running the real e2e ? I **pick rallies, watch, and download** ? **completely from the UI**.
22
23    **Outcome ("good")**: an operator signs in, picks a source, watches it index, curates
rallies, downloads a reel ? on the **live CPU droplet today** (Gemini/stub), and on a **GPU host
later** (native WASB) with **no UI fork**.
24
25    **Read to get current**: `PER_RALLY_SELECTION.md` ($3/$4/$10 contract anchors), engine
`HOSTING_PLAN.md` (gating), `DECOUPLED_COMPUTE/HANDOFF.md` (what's built),
`VIDEO_LOCALITY_MODEL.md` (privacy/retention).
26
27    ## 2. Decisions locked
28    | # | Decision | Source / date | Implication |
29    |---|---|---|---|
30    | D1 | Reverse-proxied **single origin** ; engine stays `127.0.0.1:8000` | `[verified
main.py:638]` ; 2026-06-20 | CORS `***` is moot from the browser ? the proxy is the boundary |
31    | D2 | **Edge auth**, zero engine auth code | `HOSTING_PLAN.md` ; 2026-06-20 | Caddy
basicauth OR cloudflared + Cloudflare Access |
32    | D3 | MVP drives the **in-process FastAPI worker** loop, `storage=local` | `[verified
job_worker.py:37, local.py:33]` ; 2026-06-20 | bucket worker = reserved scale path, out of MVP |
33    | D4 | Embed **RallyPicker** from `PER_RALLY_SELECTION.md` **verbatim** | PR [#11] ;
2026-06-20 | the console is a superset; do not redesign the picker |
34    | D5 | Console lives in the khelsutra **web/** React scaffold | `web/README.md` ;
2026-06-20 | Cloudflare-Pages-style root, not engine static |
35    | D6 | Compute placement is a **setting**, rendered from `GET /api/capabilities` |
`[design]` ; 2026-06-20 | UI offers ONLY what the box can honestly do |
36    | D7 | **Ingest is a source menu** (local upload · bucket URI · Drive) normalizing to
one video object | `[verified backend/storage/ref.py parse_uri]` ; 2026-06-20 | Drive is a
**stub** today ? scoped honestly ($4.2) |
37    | D8 | GPU host is **provider-agnostic** (DO droplet / GCE VM / Cloud Run+GPU) ; storage
stays the bucket | Owner 2026-06-20 | compute and storage are separate planes; no provider lock-
in |
38    | D9 | **Observability is first-class** ? structured logs on every new path + a DoD item

```

```

| Owner 2026-06-20 | see S5; verified during CUJ replay |
39 | D10 | Delivery = a **self-hosted webserver + UI the user spins up** (NOT a desktop
app); the user points at **local storage + GPU/CPU OR cloud locations** and the UI drives the
tool | Owner 2026-06-20 | **supersedes `DESKTOP_APP_PLAN.md`**; one artifact (this appliance)
runs on the owner box, a droplet, or a GPU host ? same UI, compute/storage are settings |
40
41 ## 3. Foundation ? what the engine gives us today `[verified this session unless
noted]`
42 - **FastAPI loop:** `POST /api/process` (202 + `job_id`) ? poll `GET /api/jobs/{id}` ?
`POST /api/query` ? `POST /api/compile` ? `GET /api/highlights/{video_id}/{filename}`
(FileResponse) `[main.py]`. **Chunked upload exists** ? `POST /api/upload/init` . `PUT
/api/upload/{id}/part/{i}` . `POST /api/upload/{id}/complete` . `DELETE /api/upload/{id}`
`[verified uploads.py:64,89,123,147]` ? but the bundled `app.js` **never calls them** (it only
hits `/api/process` + `/api/compile` `[verified app.js:88,373]`). So browser upload is **client-
wiring only**, no engine change.
43 - **In-process worker:** one module-global `ThreadPoolExecutor(max_workers=1)`
`[verified job_worker.py:37]` ? strictly **one job at a time**; the DB `jobs` table is the
clock; `fail_stale_jobs` sweeps queued/running on boot. Gemini chunk concurrency is a
**separate** knob `indexing.ai_max_workers` (**default 5** `[verified models.py:183]`; set to 1
to serialize) ? it is **not** already 1.
44 - **Storage:** `storage.backend` default = `"local"` `[verified models.py:298]`;
`LocalStorage.fetch_to_local` is **identity** `[verified local.py:33]`, `put` no-ops on
`src==dst` ? so on a single box the worker's pull?pipeline?push **collapses to read-local/write-
local**. URI scheme `parse_uri` `[verified ref.py]`: bare`local://` ? local; `s3://`
(AWS/R2/**DO Spaces**/MinIO, by endpoint) ; `gcs://`/`gs://` ; `gdrive://` ? **follow-up stub
(emulated only)**. Credentials never in config ? boto3 default chain /`~/aws` `[verified
s3.py]`.
45 - **Compute seam:** `indexing.detector_impl` ? `{auto`=WSL, `native`=Linux-native,
`stub`=CPU mock}; default "auto" `[verified models.py:129]`, resolved in
`trajectory_hybrid._resolve_runner` `[verified :76-85]`. **Native WASB is in-flight on branch
`feat/native-wasb-runner`** `native_wasb_runner.py` **exists** `[verified]`;
**`fusion_hybrid._build_native_runner` returns `NativeWasbRunner`** `[verified
fusion_hybrid.py:86]` while **`trajectory_hybrid._build_native_runner` still raises** `[verified
:57-63]`. Acceptance gate = **owner byte-parity** (WSL CSV == native CSV) on a real GPU box ?
not exercisable on the droplet or in the offline suite. Treat native as **not production-
verified**.
46 - **Worker CLI:** `python -m backend.worker {infer|train}` pulls a `--video-uri` ?
pipeline ? pushes `outputs/<video_id>/{intervals.csv,report.json}` `[verified
worker/__main__.py:88,149]`. The validated GPU-free/key-free combo: `detector_impl=stub` +
`--set indexing.skip_ai_handoff=true` + `storage.s3`.
47 - **Gating:** cloudflared Tunnel (outbound-only, no inbound ports) + Cloudflare Access
email-OTP; backend trusts `Cf-Access-Authenticated-User-Email` `[HOSTING_PLAN.md]`. `GET
/api/health` **now EXISTS** (engine M1, [#268]) ? `{status, sport, default_segmenter}`
`[verified main.py:127, corrected 2026-06-21 ? was "does not exist"]`.
48 - **Droplet reality:** `168.144.126.176`, **CPU-only, NO GPU**, region `blr1`; co-hosts
the marketing site (`/srv/khelsutra-site`, node `:8787`) **and** the engine (`~/rally`); ran the
full train+infer e2e against DO Spaces with the **GPU mocked** `[SETUP_NEW_MACHINE.md,
DECOUPLED_COMPUTE/HANDOFF.md]`. Shared testbed: MinIO `9000/9001` + GCS emulator `9023` also run
there.
49 - **Absent (net-new):** `GET /api/capabilities`, `GET /api/videos` (only single-row
`get_video(id)`), `GET /api/reels/{video_id}`, any source/proxy streaming endpoint, any
**Dockerfile**, any per-video **cost ledger** `[verified ? grep returns none]`.
50
51 ## 4. Design / architecture
52
53 ### 4.1 Topology & request path
54 ONE box: **edge gate** (cloudflared+Access or Caddy basicauth) ? **reverse proxy /
static host** serving the `web/` SPA bundle and proxying `/api/*` ? engine **FastAPI
`127.0.0.1:8000`** ? **LocalStorage** ? **detector seam** (Gemini/stub today; native WASB on a
GPU host). The firewall blocks inbound `8000` + the test ports `9000/9001/9023`. The marketing
site stays a **separate** systemd unit on a **separate** subdomain.
55
56 ```
57 PUBLIC INTERNET
58 ? (1) EDGE GATE ? the ONLY ingress; auth lives HERE, not in the app

```

```

59      ?      cloudflared Tunnel + Cloudflare Access (no inbound ports) OR Caddy :443 +
basicauth
60      ?
61      ?????????????????????????????????????????????????????????????????????
62      ? ONE BOX   firewall BLOCKS inbound 8000 + 9000/9001/9023      ?
63      ? (2) reverse proxy / static host (Caddy/cloudflared)      ?
64      ?      /, /assets/* ? web/ React SPA      /api/*, /api/highlights/* ? :8000 ?
65      ?      ? ONE ORIGIN to the browser ? engine CORS '*' is irrelevant ?
66      ?      ?      ?      ?      ?      ?      ?      ?      ?      ?      ?      ?
67      ? (3) ENGINE FastAPI (host=127.0.0.1:8000, HARD-bound [main.py:638])      ?
68      ?      control: jobs table + ThreadPoolExecutor(max_workers=1) drain      ?
69      ?      data: uploads.py chunked upload · /api/compile · /api/highlights      ?
70      ?      ?      ?      ?      ?      ?      ?      ?      ?      ?      ?      ?
71      ? (4) STORAGE = LocalStorage (default) fetch=identity · put=no-op(src==dst) ?
72      ?      ?      ?      ?      ?      ?      ?      ?      ?      ?      ?      ?
73      ? (5) COMPUTE seam: gemini (cloud, key) · stub (CPU mock) [droplet]      ?
74      ?      native WASB (GPU host) · auto (owner Win+WSL)      ?
75      ?????????????????????????????????????????????????????????????????????
76      RESERVED SCALE PATH (out of MVP): console "compute target" selector ? storage=s3 (DO
Spaces)
77      ? SEPARATE GPU host runs `python -m backend.worker infer --video-uri s3:///` ?
outputs/<vid>/ back
78      (needs a NET-NEW orchestration endpoint reflecting outputs into the jobs/poll
contract)
79      `
80
81      ### 4.2 Ingest sources (the headline) ? one menu, one normalized video object
82      All sources normalize to ***"a video the pipeline can read"*** ? a local path (MVP) or a
`StorageRef` (scale):
83
84      | Source | Path today | Status |
85      |---|---|---|
86      | **Local upload** | SPA chunks the file ? `max_part_mb` via the existing
`/api/upload/init\|part\|complete` `[verified uploads.py]` ? server path ? `POST /api/process`.
(Scale: also `StorageBackend.put` to `s3:///gcs://`) | **wire client only** (endpoints exist)
|
87      | **Bucket URI** (`s3://`, `gs://`) | **`POST /api/process` accepts the bucket URI
directly** (resolve ? fetch to scratch ? same pipeline ? API DB) `[verified main.py:159,274,283
? corrected 2026-06-21; was "net-new"]`; the worker CLI also fetches `--video-uri`. So the
**engine endpoint is NOT net-new** ? only the SPA call that POSTs the URI + polls `/api/jobs`
is. *(Default single-user mode; HOSTED mode w/ `allowed_video_root` correctly rejects a bucket
URI 400.)* | **engine: DONE (verified); SPA ingest call: design (= B-P2)** |
88      | **Google Drive** (`gdrive://`) | `parse_uri` recognizes it, but the gdrive backend is
an **emulated stub** `[verified ref.py, DECOUPLED_COMPUTE research]` | **stub ? roadmap;** MVP
requires a Drive?bucket pre-step; do **not** present Drive as live until the backend is real |
89
90      ### 4.3 Compute placement matrix (one UI, surfaced from `/api/capabilities`)
91      | Placement | Runs | Needs | Real/mock | Honest caveat |
92      |---|---|---|---|---|
93      | **CPU droplet ? gemini** (MVP target) | `segmenter=gemini`, in-proc worker |
`GEMINI_API_KEY`; network; **no GPU** | **REAL** | Cloud brain: uploads ?1280px chunks to Google
? **must be in consent copy** ; **no per-video cost ceiling** today |
94      | **CPU droplet ? stub** | trajectory hybrid + `detector_impl=stub`+`skip_ai_handoff` |
nothing | **MOCK** | not real tracking ? UI must label "stub (CPU mock ? not real detection)";
demo/regression only |
95      | **CPU droplet ? motion** | `segmenter=motion` | nothing | REAL, low-quality | only no-
key/no-GPU real segmenter; never surface `server/receiver/events` (fabricated) |
96      | **GPU host ? native WASB** (in-flight) | `wasb`+`detector_impl=native` ?
`NativeWasbRunner` (in-proc torch) | a **GPU host ? DO GPU droplet / GCE GPU VM / Cloud
Run+GPU** ; `WASB_WEIGHTS` | **REAL** , on-device | partially wired (fusion builds it, trajectory
refuses); **owner byte-parity gate** ; **Cloud Run/GCE need a containerized worker ? no
Dockerfile exists, and WASB's torch-1.11 conda env makes the image non-trivial** |
97      | **Owner Win+WSL ? auto** | `wasb`+`detector_impl=auto` (WSL2+conda) | Windows+WSL2+GPU
| REAL (today) | Windows-only transport; the working real-GPU path for owner verification |
98      | **Remote GPU worker via bucket** (reserved) | console CPU + `storage=s3`; separate GPU

```

host runs `backend.worker infer` | bucket creds; a GPU host; a **net-new** orchestration endpoint** | REAL (remote) | FastAPI loop and worker CLI are **separate** today; "remote GPU as a setting" is **design** ? selector shown **disabled** in MVP |

99

100 **Provider note:** **Cloud Run + GPU** (scale-to-zero, pay-per-job) is the strongest fit for the engine's own **per-video-billing** economics (`DECOUPLED\_COMPUTE` #246) ? gated on the **containerization feasibility check** above.

101

102 **4.4 Auth / exposure (non-negotiable invariant)**

103 No engine auth, by design; the gate is external. **Invariant** (all three must hold):** engine bound to `127.0.0.1` + firewall closes `8000` + edge gate up. Go-live checklist
 `[HOSTING\_PLAN.md:102-108]`: Access/basicauth ON; **rotate** the chat-shared `GEMINI\_API\_KEY`**;
 set `security.allowed\_video\_root`; rate-limit; per-user quota.

104

105 **4.5 UI operator console (web/ SPA ? superset of RallyPicker)**

106 - **S1 Ingest & Process** ? the source menu (§4.2) + a segmenter picker from
 `/api/capabilities` ? `POST /api/process`.

107 - **S2 Progress** ? 3s poll of the free-text stage
 (`hashing/validating/segmenting/finalizing`), a **"one job at a time"** notice, and the restart-failure terminal state.

108 - **S3 Rally pick ? reel** ? **embed RallyPicker verbatim** (`PER\_RALLY\_SELECTION.md`):
 honor `min\_shot\_count`; never render motion `server/receiver/events` or a fake confidence meter.
Per-rally is ONE screen of the console.**

109 - **S4 Reels / History** ? needs `GET /api/videos` + `GET /api/reels/{video\_id}` for
reload-survival.

110 - **S5 Settings** ? render `config` + `capabilities`; the compute/storage **target**
 selector shown **DISABLED** ("single-box: local") until the bucket-worker bridge lands.

111

112 **4.6 Reuse map**

113 **Reused (no re-plumb):** the full `/api` loop; the in-process worker; the chunked
 upload endpoints (un-called); LocalStorage default; the detector seam incl. the landing
 `NativeWasbRunner`; the **RallyPicker** design + anchors; the `web/` scaffold; the
 `HOSTING\_PLAN` gate; the `SETUP\_NEW\_MACHINE` runbook. **Net-new:** Caddyfile + systemd units;
 `GET /api/capabilities`; `GET /api/videos`; `GET /api/reels/{video\_id}`; `GET /api/health`; the
 SPA console screens; ingest-from-bucket wiring; a retention sweep; (reserved) the bucket-worker
 bridge.

114

115 **5. Observability ? logging & monitoring** `[required, first-class]`

116 Launch-readiness directive (owner 2026-06-20): **logs and monitoring taken seriously**,
 with a full log-analysis pass across every new code path.** The engine already uses a
 centralized Python logger `[verified main.py:12-17]` ? carry that discipline into every new
 path.

117

118 - **Correlation ID end-to-end.** Thread the existing `job\_id` (from `/api/process`)
plus an `upload\_id`/`request\_id` through ingest ? process ? jobs ? compile ? highlights, and
 into the worker for the bucket path, so **one CUJ is traceable** across UI ? FastAPI ? worker ?
 GPU host. Log it on every line.

119 - **Structured logs.** New endpoints/screens emit structured (JSON-able) logs at **entry**
 / result / error ? not just "it ran". The SPA logs client-side errors + failed fetches with
 the same correlation id.

120 - **Loud failures, never silent drops.** Carry the engine's ethos (#240 "surface failed
 chunks loudly") + RallyPicker's `min\_shot\_count`-warning rule: a partial/empty reel must
never look complete; every dropped clip/chunk is logged + surfaced in the UI.

121 - **Health + metrics.** Ship the missing `GET /api/health` `[HOSTING\_PLAN.md:42]` and a
 thin metrics surface (jobs queued/running/failed, last error, disk free, per-video cost once a
 ledger exists) for the engine-mode banner + alerting.

122 - **Aggregation for transient hosts.** A scale-to-zero Cloud Run / GPU host is
ephemeral ? logs must ship to a durable sink (the bucket or a log service), not die with the
 instance.

123 - **Log-analysis pass before merge (DoD item).** Every PR that adds a code path includes
 a check that the path logs meaningfully, and the **logs are inspected during CUJ live-replay**
 (§8 Layers 1?3). Ties [[feedback-launch-quality-bar]].

124

125 **6. Risk table**

126 | Risk | Today | Mitigation |

```

127 |---|---|---|
128 | No app auth + CORS `` `[main.py:42]` | open if any of {localhost-bind, firewall,
gate} slips | enforce all three; **LIVE posture test** that raw `:8000` is refused from outside
|
129 | Test ports on shared droplet | MinIO `9000/9001` + GCS emu `9023` | firewall closes
them |
130 | Retention | `output/`+`uploads/` originals **never deleted** | sweep / keep-proxy-only
before any non-owner exposure |
131 | Surprise Gemini/GPU bill | **no cost ceiling/ledger** | per-video cap;
`ai_max_workers` (default 5) tunable; `max_workers=1` job serialization |
132 | Native WASB unverified | partially wired; owner-parity-gated | treat as not-
production-verified until owner byte-parity on a real GPU box |
133 | New engine endpoints | `capabilities`/`videos`/`reels`/`health` don't exist |
coordinate engine PRs with active sessions; **branch from `origin/master`, re-verify HEAD,
explicit `git add`** |
134 | Drive ingest overstated | `gdrive://` is a stub | don't present Drive as live; require
Drive?bucket pre-step until real |
135 | No Dockerfile | runbook-only reproducibility | feasibility-check a worker image before
promising Cloud Run/GCE |
136 | Shared-checkout collisions | active worker sessions share the engine tree | the
branch-safety protocol on every engine-side PR |
137 | **Provisioning-credential leak** | a DO API token / cloud creds could leak if pasted
in chat or committed (cf. the **already-rotated Spaces key** exposed in chat,
`DECOUPLED_COMPUTE/HANDOFF.md`) | authenticate the CLI **on-box** (`doctl auth init` interactive
/ `gcloud` already authed) so the secret never enters chat or the repo; least-privilege token,
revoke after; creds 0600, never committed; **creating paid GPU infra is a confirm-first action
every time** |
138
139 ## 7. Honest limits ? what this does NOT do
140 Does **not** add multi-tenant identity/RLS (single-tenant alpha; uploads/outputs
global). Does **not** run real WASB on the CPU droplet (Gemini or stub only). Does **not** make
the remote-GPU bucket path work (design; selector disabled). Does **not** provide in-UI source
playback, job cancel, or a true progress bar (coarse free-text stage, poll-only). Does **not**
containerize (runbook; no Dockerfile `[verified]`). Does **not** verify native WASB (owner byte-
parity on a real GPU box, not this offline suite). A green SPA + contract tests prove the
**console plumbing**, not engine rally-detection quality (a separate track).
141
142 ## 8. CUJs & live-test plan `[required]`
143 **CUJs:** **CUJ-1** ingest (Drive/bucket/local) ? index ? reel ? download (core);
**CUJ-2** re-open a past match after reload; **CUJ-3** pick-by-query (deterministic filters +
honesty-hiding); **CUJ-4** settings/compute placement (truthful capabilities).
144
145 **Live-test plan ? 5 layers, every CUJ replayed LIVE throughout development:**
146 - **L1 ? Mock engine (hermetic, fast, default + CI).** A tiny FastAPI/MSW implementing
the **exact** contract (upload; `process` 202; `jobs` state machine incl. restart-failure;
`query` with a motion fixture for honesty-hiding; `config`; `capabilities` per box profile ?
CPU-no-key / CPU-with-key / GPU; `compile`; `highlights` tiny mp4; the new `videos`/`reels`).
The SPA drives **all 4 CUJs in <1s**, **replayed via the Claude Preview/browser tools** (start
dev server ? fill the ingest picker ? click through process/poll/RallyPicker ? assert the reel
link via snapshot/network). Catches reload-survival (CUJ-2), the `play_segments` field-name bug
(CUJ-3), capability honesty (CUJ-4).
147 - **L2 ? Engine-contract checks (real FastAPI, GPU-free).** Boot `python -m backend.main
--server` with `storage=local` + `detector_impl=stub` + `skip_ai_handoff` and assert the **mock
and the real engine agree shape-for-shape** (202 keys, jobs enum, `result.play_segments` schema,
compile `{status,filepath}`, highlights content-type). Every new endpoint is born with a mock
handler **and** a real-engine contract test **in the same PR**. Runs in CI.
148 - **L3 ? Real local e2e (the MVP substrate).** Through the gate on the droplet: CUJ-1
with `stub` (key-free) and `gemini` (real, short clip to bound cost); the **exposure-safety
posture test** (raw `:8000`/`:900x` refused from outside) as an assertion, not an assumption;
reuse the `parity.yml` dual-host pattern.
149 - **L4 ? CI regression (green gates merge).** `npm test` + a stub-engine contract job +
headless browser-replay CUJ scripts; one small PR per tracker row off `origin/master`, **no
stacking** ; coverage at/above the floor; **logs inspected during replay** ($5).
150 - **L5 ? Owner hardware e2e (handed off).** Native WASB **byte-parity** on a real GPU
box; real-4K **golden-corpus** runs (Gemini + native); Modern-Standby/cv2-vs-ffmpeg`

```

behavior; a multi-GB browser upload. Reported back into S9, stated plainly as ****not verified**** in the offline suite.

151

152 **## 9. Deliverables & progress tracker ? **source of truth****

153 Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. ****One small PR per row****, branched from `origin/master`, no stacking; CI green gates merge; coverage ? floor; each PR ships its tests + meaningful logs (S5).

154

155 | ID | Deliverable | Depends on | Gated? | Status | PR |

156 |----|-----|-----|-----|-----|----|

157 | T0 | This design doc + `docs/README.md` + `NEXT\_STEPS.md` pointers | ? | owner | ? |
(this PR) |

158 | T1 | Mock engine + capability fixtures + browser-replay CUJ scripts (L1) | T0 | no | ? |
| ? |

159 | T2 | Appliance shell: Caddyfile + systemd units; firewall; ****edge auth****; rotate key (P1) | ? | owner | ? | ? |

160 | T3 | L3 live posture test (raw `:8000`/`:900x` refused from outside) | T2 | no | ? | ? |
|

161 | T4 | S1 ingest (local upload wired) + segmenter picker | T1 | no | ? | ? |

162 | T5 | S2 progress (poll + one-job + restart-failure) | T4 | no | ? | ? |

163 | T6 | S3 embed ****RallyPicker**** + compile + download | T4 | no | ? | ? |

164 | T7 | ***(engine)*** `GET /api/capabilities` + `GET /api/health` + banner | T1 | coord | ? |
| ? |

165 | T8 | ***(engine)*** `GET /api/videos` + `GET /api/reels/{video\_id}` (S4 reload-survival) |
? | coord | ? | ? |

166 | T9 | S5 Settings (config+capabilities; disabled compute/storage target) | T7,T8 | no |
? | ? |

167 | T10 | Observability pass: correlation id end-to-end + structured logs + metrics (S5) |
T4 | no | ? | ? |

168 | T11 | Retention sweep for `output/+`uploads/` | ? | owner | ? | ? |

169 | T12 | Ingest from ****bucket URI**** (`s3:///`gs://`) wiring ? ****engine half DONE****
(`/api/process` accepts URIs, M2/[#280], `[verified]`); remaining = the SPA ingest call (= ****B-P2****) | T4 | coord | ? | engine done; SPA pending |

170 | T13 | ****GPU host stand-up**** (provider-agnostic) + wire `detector\_impl=native`; owner
byte-parity (P4) | branch | ? owner | ? | ? |

171 | T14 | ***(reserved)*** Bucket-worker bridge ? remote-GPU "compute as a setting" (P5) |
T8,T12 | ? owner | ? | ? |

172

173 ****Owner priority (2026-06-20):** T13 (GPU host) is elevated to run in parallel from the
start ? owner provisions the host + token; the engine session lands native WASB; the UI MVP
does ****not**** block on it (ships on Gemini).

174

175 **## 10. Open questions ? owner / external**

176 ****Blocking gates:**** (1) ****edge-auth**** ? Caddy basicauth (self-contained) vs
cloudflared+Cloudflare Access (matches the two-host design, no inbound ports)? (2) ****retention
defaults**** ? auto-delete originals after compile? keep proxies+intervals only? window?
(privacy/consent). (3) ****per-video Gemini/GPU cost ceiling**** ? what cap, hard-fail vs warn? (4)
****GPU host choice**** ? DO GPU droplet / GCE GPU VM / Cloud Run+GPU (and is a containerized worker
worth building for Cloud Run)? (5) ****P5 bridge ownership**** ? who builds the orchestration
endpoint + how it coordinates with the active worker sessions.

177 ****Nice-to-knows:**** console home (web/ vs engine static); a review-playback streaming
endpoint; per-user scoping timing; containerization vs runbook.

178

179 **## 11. Definition of done**

180 - [] CUJ-1 works end to end on the droplet (Gemini ****and**** stub) behind the gate, via
the ingest menu.

181 - [] Raw `:8000`/`:900x` unreachable from outside (****tested****, not assumed).

182 - [] Local upload wired; bucket-URI ingest at least designed/tracked; Drive scoped
honestly (not faked).

183 - [] RallyPicker embedded with honesty rules; `min\_shot\_count` surfaced.

184 - [] `/api/capabilities` + `/api/health` + banner truthful; reload-survival via
`/api/videos` + `/api/reels`.

185 - [] ****Observability:**** correlation id threaded end-to-end; structured logs at every
new path; loud failures; logs inspected during CUJ replay.

186 - [] Retention sweep in place; `GEMINI\_API\_KEY` rotated; `allowed\_video\_root` set.

```

187     - [ ] CI green (L2 contract + L4 regression); coverage ? floor; $9 tracker all
188     ??-resolved.
189
190     ## 12. References
191     **Engine code `[verified this session]`:**
192     `backend/main.py:12-17,42,52,307-328,331-340,342-396,638`;
193     `backend/api/uploads.py:64,89,123,147`; `backend/api/job_worker.py:37`;
194     `backend/api/static/app.js:88,344,373`;
195     `backend/storage/{base.py,local.py:33,ref.py,__init__.py:43}`;
196     `backend/config/models.py:129,183,298`;
197     `backend/pipeline/segmenters/{trajectory_hybrid.py:57-63,76-85, fusion_hybrid.py:76-86,
198     gemini.py:470-485, motion.py:191-198}`; `backend/pipeline/detectors/{native_wasb_runner.py,
199     stub_runner.py}`; `backend/worker/__main__.py:88,98,149`; `config.json` (`default_segmenter:
200     gemini`). **Engine docs:** `HOSTING_PLAN.md:42,102-108`; `VIDEO_LOCALITY_MODEL.md`;
201     `DESKTOP_APP_PLAN.md`; `SETUP_NEW_MACHINE.md`; `PLATFORM_ARCHITECTURE.md`;
202     `DECOUPLED_COMPUTE/{README,02,04,07,HANDOFF}.md`; `UI_PROPOSAL.md:41-66`;
203     `PROJECT_DOC_TEMPLATE.md`. **This repo:** `PER_RALLY_SELECTION.md`, `REQUEST_ACCESS_FORM.md`,
204     `PORTABILITY.md`, `web/README.md`.
205
206     ---
207
208     ### Changelog
209
210     - 2026-06-20 ? Initial `DRAFT`. Consolidated three single-box-appliance designs
211     (FastAPI-native / unified-origin / decoupled-profile) into the reverse-proxied unified-origin
212     spine + compute-as-a-setting framing, via a multi-agent design pass; every engine-fact anchor
213     re-verified against source before writing (catching the native-WASB partial-wiring +
214     `ai_max_workers` default nuances). Folded in the owner's ingest-menu (Drive/bucket/local),
215     multi-provider GPU, and first-class Observability directives.
216
217     195

```

24. assistant (2026-06-21T13:52:43.474Z)

This is rich. Now let me read the HOSTING_PLAN.md (the M9 auth/Cloudflare Access design) and check the M9 auth branch state, plus the GCP create_gpu_vm.sh and teardown.sh.

25. user (2026-06-21T13:52:44.438Z)

```

1      # Hosting Plan ? khelsutra.guru
2
3      **Status:** PROPOSAL ? awaiting owner ratification.
4      **Date:** 2026-06-10 · **Constraints:** budget $10?25/mo · GCP credits available ·
5      DigitalOcean
6      acceptable · alpha backend = owner's GPU box behind a tunnel · 10?25 alpha testers ·
7      registrar = GoDaddy.
8
9      > **? Update (2026-06-20, ADR-013): DigitalOcean is DROPPED for COMPUTE** ? GCP is the
10     sole compute
11     > stack (training + serving). The ***"All-DigitalOcean"*** compute option below is
12     retired; DO Spaces
13     > remains only a *possible* S3-compatible storage target. Edge/site hosting choices here
14     are unaffected.
15     **Companion:** `docs/UI_PROPOSAL.md`.
16
17     ## 1. Recommendation in one line
18
19     **Alpha: free edge + your box** (Cloudflare DNS/Pages/Access/Tunnel ? FastAPI on the GPU
20     box) at
21     **$0/mo**, saving GCP credits; **Beta: GCP control plane + your box as pull-worker**
22     (Cloud Run +
23     GCS + Firebase Auth, credits-covered), keeping the edge where it's free.
24
25     Rationale for the Cloudflare pieces despite the GCP preference: GCP has no free
26     equivalent of the
27     Tunnel + Access combo (IAP requires a load balancer, ~$18/mo standing cost), and the
28     alpha needs
29     exactly three things GCP charges for but Cloudflare gives free: a secure tunnel to a

```

```

home box with
21     a custom domain, email-allowlist auth (free ?50 users ? fits 10?25 testers), and static
hosting.
22     Credits are better spent at beta on compute/storage, which is where GCP is excellent.
23
24     ## 2. Alpha topology ($0/mo)
25
26     ```
27     tester browser
28         ? https
29         ?
30     Cloudflare edge (free): DNS + TLS + Access (email-OTP allowlist)
31         ??? app.khelsutra.guru ? Cloudflare Pages (React SPA; auto-deploy from GitHub)
32         ??? api.khelsutra.guru ? Cloudflare Tunnel (outbound-only `cloudflared` service on
the box)
33         ?
34         ?
35         FastAPI :8000 on the GPU box (Windows)
36         ? Gemini segmenter path (no GPU in request path)
37         ? async jobs (#16), SQLite WAL, output/ on disk
38     ```
39
40     - **No inbound ports opened** on the home network; `cloudflared` dials out. Home IP
never exposed.
41     - **Auth:** Cloudflare Access in front of BOTH hostnames; backend trusts the
42         `Cf-Access-Authenticated-User-Email` header (verify the Access JWT for rigor) for per-
user scoping.
43     - **Upload limit:** free-tier proxy caps request bodies at **100 MB** ? SPA uploads in
?90 MB
44         chunks (UI_PROPOSAL B2). Match videos (1?4 GB) flow fine as chunks.
45     - **Box offline = service offline.** Acceptable for alpha; SPA shows a friendly
"processing engine
46         offline" banner (a `/api/health` ping). Beta removes this with the queue.
47     - **Costs:** Pages, Tunnel, Access, DNS = $0. Only the Gemini per-run cost
(~$0.05/match) and
48         domain renewal.
49
50     ## 3. Beta topology (GCP credits; ~$15?20/mo post-credits)
51
52     ```
53     browser ??? Cloudflare edge (free, unchanged) ??? app.khelsutra.guru (Pages)
54                                     ??? api.khelsutra.guru ??? Cloud Run
(FastAPI control plane, scale-to-zero)
55                                     ? jobs
table (Cloud SQL Postgres)
56                                     ? signed
resumable URLs
57                                     ?
58                                     GCS buckets
(uploads / reels, lifecycle ? cold)
59                                     ? pull
jobs, outbound-only
60                                     GPU box = first
`byo_worker` (PLATFORM P3)
61     ```
62
63     - Uploads go **browser ? GCS directly** (resumable signed URLs) ? no proxy body limit,
no home
64         bandwidth for ingest; the box pulls only what it processes (WASB/owned-model jobs),
Cloud Run
65         handles Gemini-path jobs entirely in-cloud.
66     - Auth graduates to **Firebase Auth** (free; Google + email sign-in; India-friendly
phone OTP
67         later); Access allowlist retires.
68     - Post-credits estimate: Cloud Run ~$0?5 (free tier: 2 M req + 180 K vCPU-s/mo, scale-

```

```

to-zero) .
69     GCS 100 GB ? $2.60 + egress $0.12/GB (reel downloads ? watch this line) . Cloud SQL
smallest
70     ? $10 ? **~$15?20/mo**, inside budget. Egress lever if it grows: serve reels from
Cloudflare R2
71     (zero egress) while keeping everything else GCP.
72
73     ## 4. Alternatives considered
74
75     | Option | Monthly (alpha ? beta) | Pros | Cons |
76     |---|---|---|---|
77     | **Recommended hybrid** (CF edge + box ? +GCP) | **$0 ? ~$0 w/ credits** (~$15?20
after) | Cheapest at every phase; credits used where they matter; maps to PLATFORM P2/P3 | Two
vendors (CF + GCP) |
78     | **All-DigitalOcean** | ~$11 ? ~$17 ($6 droplet + Spaces $5: 250 GiB + **1 TiB egress
included**) | One simple vendor; Spaces' included egress is the best video-download deal; per-
second billing | No credits; no scale-to-zero; still needs a tunnel-equivalent to reach the box
(run `frp`/WireGuard on the droplet) |
79     | **All-GCP from day 1** | ~$18+ ? credits | Single vendor, credits immediately | LB+IAP
standing cost just to gate an alpha; no free tunnel to the home box; credits burn on plumbing |
80     | **All-Cloudflare** (Pages+Workers+R2) | ~$0 ? ~$5 | Zero egress on R2; cheapest
forever | FastAPI doesn't run on Workers (API stays on box/Cloud Run anyway); ignores GCP
credits |
81
82     ## 5. Domain setup ? GoDaddy ? Cloudflare DNS (one-time, ~15 min, you)
83
84     1. Create a free Cloudflare account ? **Add a domain** ? `khelsutra.guru` ? Free plan.
Cloudflare
85         imports existing records and shows **two nameservers** (e.g.
`xxx.ns.cloudflare.com`).
86     2. GoDaddy ? **My Products ? khelsutra.guru ? Manage DNS ? Nameservers ? Change ?
87         "I'll use my own nameservers" ? paste the two Cloudflare nameservers ? Save.
Propagation:
88         minutes to 24 h. (Registration/renewal stays at GoDaddy; only DNS moves. Optional
later:
89         transfer the registration to Cloudflare Registrar for at-cost .guru renewals.)
90     3. Records (I manage after takeover): `app` ? Pages project; `api` ? `<tunnel-
id>.cfargotunnel.com`
91         (created automatically by `cloudflared`); apex redirect ? `app`.
92
93     ## 6. What I need to take over management
94
95     | Access | Scope | Used for |
96     |---|---|---|
97     | **Cloudflare API token** (you create: dash ? My Profile ? API Tokens) | Zone:DNS>Edit,
Account:Pages>Edit, Account:Access:Apps>Edit, Account:Tunnel>Edit ? scoped to this zone | DNS
records, Pages deploys (wrangler), Access policies, tunnel config |
98     | **GitHub repo** `khelsutra` (private) | Pages connects once (dashboard, you; build
root = `web/`) ? every push auto-deploys | I write the SPA in your Projects folder; commits
pushed = deployed |
99     | **On the box** (one-time, I hand you exact commands) | `cloudflared` installed as a
Windows service + `config.yml` | The tunnel itself |
100    | **GCP (beta only)** | A `khelsutra` project + service account (Cloud Run admin,
Storage admin, Cloud SQL admin) | Control-plane deploys |
101
102    Day-to-day flow after setup: I build ? commit ? push ? Pages deploys the SPA
automatically;
103    API/tunnel/DNS changes via the scoped token. Secrets (Gemini key, Access policy) never
live in the
104    repo; `.env` on the box + Cloudflare/GCP dashboards.
105
106    ## 7. Go-live security checklist (alpha)
107
108    Access allowlist ON for both hostnames ? CORS narrowed to `https://app.khelsutra.guru` ?
rate

```

```

109     limit + per-user upload quota ? `security.allowed_video_root` set ? upload size cap +
extension
110     allowlist ? **rotate the Gemini API key** (NEXT_STEPS housekeeping ? it was shared in
chat) ?
111     confirm no router port-forwards ? weekly backup: SQLite DB + `output/` ? GCS bucket
(rclone
112     scheduled task; credits).
113
114     ## 8. Decisions needed from owner
115
116     1. Ratify the **hybrid recommendation** (CF free edge now, GCP credits at beta)? Or
force
117         single-vendor (DO / GCP) despite the cost notes above?
118     2. OK to move **DNS to Cloudflare** (registration stays at GoDaddy)?
119     3. Beta reel storage: **GCS (credits)** vs **R2 (zero egress)** vs **DO Spaces (1 TiB
included)** ?
120         can defer until beta Drop 2.
121     4. Approve the access grants in $6 so I can run the setup end-to-end.
122
123     *Ratified choices get recorded in `docs/archives/decisions/DECISIONS.md`.*
124

```

26. user (2026-06-21T13:52:44.681Z)

```

1     #!/usr/bin/env bash
2     # Create a rally GPU VM that ALWAYS installs the Ops Agent (Observability). This is THE
one way to
3     # spin up a rally GPU box ? it bakes the ops-agent startup-script into the VM metadata,
attaches the
4     # rally-worker service account (+ cloud-platform scope so it can read/write the bucket +
report
5     # metrics), and uses a driver-ready Deep-Learning-VM image (nvidia-smi works
immediately).
6     #
7     # It sweeps zones for GPU capacity ? asia-south1 (Mumbai, co-located with the bucket)
first, then
8     # asia-southeast1 (Singapore), which L4 frequently falls back to when Mumbai is stocked-
out.
9     #
10    #   bash deploy/gcp/create_gpu_vm.sh                                # L4 (default), name
rally-gpu
11    #   RALLY_VM_NAME=rally-m7 RALLY_GPU=t4 bash deploy/gcp/create_gpu_vm.sh
12    #
13    # Prereq (once per project): bash deploy/gcp/provision.sh   (enables logging/monitoring
+ grants the
14    # SA the Ops-Agent writer roles) AND the GPUS_ALL_REGIONS quota >= 1 (README $2).
15    #
16    # ? DELETE when done (a running GPU VM is the only real cost):
17    #   RALLY_GCP_ZONE=<zone-it-printed> bash deploy/gcp/teardown.sh delete
18    set -uo pipefail
19
20    PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
21    NAME="${RALLY_VM_NAME:-rally-gpu}"
22    SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
23    GPU="${RALLY_GPU:-l4}"      # l4 | t4
24    HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
25    STARTUP="$HERE/ops-agent-startup.sh"
26    export CLOUDSDK_CORE_DISABLE_PROMPTS=1
27
28    ZONES="${RALLY_GCP_ZONES:-asia-south1-a asia-south1-b asia-south1-c asia-southeast1-a
asia-southeast1-b asia-southeast1-c}"
29
30    if [ "$GPU" = "l4" ]; then
31        MACHINE="g2-standard-4"; ACCEL=( )      # L4 is integral to the g2 machine family
32    else

```

```

33     MACHINE="n1-standard-8"; ACCEL=(--accelerator=type=nvidia-tesla-t4,count=1)
34 fi
35
36 ERR="$(mktemp)"
37 for Z in $ZONES; do
38     echo "== trying $NAME ($GPU) in $Z =="
39     if gcloud compute instances create "$NAME" --project="$PROJECT" --zone="$Z" \
40         --machine-type="$MACHINE" "${ACCEL[@]}" \
41         --maintenance-policy=TERMINATE --provisioning-model=STANDARD \
42         --image-family=common-cu129-ubuntu-2204-nvidia-580 \
43         --image-project=deeplearning-platform-release \
44         --boot-disk-size=120GB --boot-disk-type=pd-balanced \
45         --service-account="$SA" --scopes=cloud-platform \
46         --metadata-from-file=startup-script="$STARTUP" 2>"$ERR"; then
47         echo "? CREATED $NAME in $Z ? the Ops Agent installs on boot (Observability ~2-3 min
later)."
48         echo "    SSH:    gcloud compute ssh $NAME --zone=$Z"
49         echo "    DELETE: RALLY_GCP_ZONE=$Z bash deploy/gcp/teardown.sh delete"
50         rm -f "$ERR"; exit 0
51     fi
52     if grep -qiE "STOCKOUT|does not have enough" "$ERR"; then
53         echo "    (no $GPU capacity in $Z ? trying the next zone)"; continue
54     fi
55     echo "    create failed:"; tail -4 "$ERR"; rm -f "$ERR"; exit 1
56 done
57 echo "? No $GPU capacity in any swept zone ($ZONES). Try again later or RALLY_GPU=t4."
58 rm -f "$ERR"; exit 1
59

```

27. assistant (2026-06-21T13:52:47.881Z)

Now let me check the M9 auth implementation in the codebase (the branch is ``feat/serving-m9-auth``), the storage sharing model, and the DECISIONS for M9. Let me also look at the config models for auth and the worker entry point.

28. user (2026-06-21T13:52:52.349Z)

```

e68b583 fix(api): resolve a bucket-ingested video's source to a local file before stitching
(#289)
43eccc1 feat(serving): M8 ? per-job cost ledger + versioned pricebook + /cost API (default-OFF)
(#288)
8cdf65c feat(serving): M6 ? Cloud Run GPU dispatch shim + worker image (default-OFF, mocked)
(#287)
b5af66e feat(serving): M5 ? truly-local profile per-profile startup checks + L4 profile-parity
test (default-OFF) (#286)
fefa412 docs(serving): resolve $8 design picks (D16/D17) + reconcile per-video P3 vs M3 (#285)
06ff4b7 test(api): lock gs://|gcs:// /api/process -> queryable API-DB video_id (B-P1 guard)
(#284)
cee4baa feat(serving): M4 ? DeviceContext + native WASB CPU-fallback (device="auto", cuda stays
strict) (#283)
2afd54f feat(serving): M3 ? configurable output/scratch base (de-hardcode output/, default-OFF)
(#282)
41d45fd fix(server): drain async jobs under `python -m backend.main` (double-module hang) (#281)
e111add feat(serving): M2 ? URI-aware input (input_backend/input_root, local=identity) (#280)
ce22791 feat(serving): M1 ? deployment.profile + compute_target config (pure, additive, default-
OFF) (#279)
f518e97 docs(serving): M0 APPROVED + lock D13-D15 (switchable profiles, Drive Northstar,
consent-not-surprise) (#278)
49c082a docs(serving): lock the 5 M0 gating decisions (D7-D12) + add the data-flywheel harness
(M11) (#277)
f592281 docs: point I18N_PLAN + UI_PROPOSAL (+ DOC_STATUS) at the khelsutra UX/l10n project
(#275)
f6f15ca docs: isolate field-ops/PMF + Roora into archives/; carve a no-ML docs/data_pipeline/
(#276)
---DIFF STAT vs master---

```

tests/test_rally_seg_eval.py	155 +++-
tests/test_rally_seg_eval_report.py	306 +++++++
tests/test_results.py	50 ++
tests/test_served_gate_a_regression.py	96 +-
tests/test_server_entrypoint.py	95 +++
tests/test_startup_checks.py	226 ++++++
tests/test_storage.py	328 +++++++
tests/test_stub_runner.py	129 ++++
tests/test_tolerance_metrics.py	188 ++++++
tests/test_tracknet_runner.py	96 +++
tests/test_training_gen0.py	56 ++
tests/test_upload_download.py	3 +-
tests/test_wasb_cache.py	20 +
tests/test_windowing.py	37 +-
tests/test_worker.py	309 +++++++
tests/test_workspace.py	122 +++
tests/test_yolo_hybrid.py	41 +
tools/generate_highlights.py	118 +-
training/gen0/harness.py	50 +-

197 files changed, 21319 insertions(+), 1368 deletions(-)

---UNTRACKED auth---

29. user (2026-06-21T13:52:54.064Z)

Found 17 files

- pyproject.toml
- docs\COMPUTE_DECOUPLED_SERVING\README.md
- backend\config\models.py
- docs\NEXT_STEPS.md
- docs\archives\decisions\DECISIONS.md
- docs\UI_PROPOSAL.md
- docs\UI_ALPHA_EXECUTION_PLAN.md
- docs\SETUP_NEW_MACHINE.md
- docs\STORAGE_SHARING_MODEL.md
- docs\HOSTING_PLAN.md
- docs\archives\past_projects\DECOUPLED_COMPUTE\03-PLATFORM-OPTIONS.md
- artifacts\appliance_design.json
- artifacts\so.json
- artifacts\so.py
- backend\storage\s3.py
- artifacts\landing_index.html
- artifacts\site_deploy\index.html

30. assistant (2026-06-21T13:52:57.665Z)

The M9 auth implementation isn't on this branch yet (the `?? backend/api/auth.py` in the snapshot was likely a prior uncommitted scratch). Let me read the COMPUTE_DECOUPLED_SERVING README (the authoritative serving plan with milestones M0-M11 including M9), and the auth-relevant config models section.

31. user (2026-06-21T13:52:58.528Z)

```

1      # Compute-Decoupled Rally Serving ? one pure core, profile-selected compute + storage
2
3      > **Status:** ? **M0 APPROVED** (owner, 2026-06-21) ? **IN PROGRESS (build M1+)** .
**Owner:** Avi . **Created:** 2026-06-21 . **Last updated:** 2026-06-21
4      > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to
`docs/archives/past_projects/` when DONE)
5      > **Tracking anchors:** $7 progress tracker is the source of truth; pointer in memory
`current-state` + `docs/NEXT_STEPS.md` once work starts.
6      > **Relation to existing docs:** the productionization successor to the archived
`DECOUPLED_COMPUTE` (M3/M4); realizes the `ComputeBackend`/`StorageBackend` registries from
[PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) $3; the **truly-local** profile is the
L0 tier of [VIDEO_LOCALITY_MODEL.md](../VIDEO_LOCALITY_MODEL.md) and the `LocalDesktop` backend

```

```

of [DESKTOP_APP_PLAN.md](../DESKTOP_APP_PLAN.md).
7     > **Naming:** this was the "Cloud Run + GPU serving subproject" (ADR-012). Renamed
**compute-decoupled serving** because **truly-local is a co-equal, first-class profile**, not an
afterthought.
8     > **Honesty note:** claims tagged `[verified]` (checked against code by the design
workflow `wf_56274ff0`), `[design]` (proposed), `[researched]`.
9
10    ---
11
12    > **? Northstar (D14):** a **mobile, app-like** flow ? point at a **Google Drive** video
? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next
to the source**. Two design promises around it: a user can **switch local?cloud anytime** (D13,
a headline feature), and execution is **never a surprise** ? auto-detect *suggests*, the user
*confirms* (D15).
13
14    ## 0. TL;DR
15    The rally engine is a **pure function of `(input bytes + config) ? outputs`** in three
deterministic stages ? **DETECT ? WINDOW ? STITCH** ? that **no deployment profile may branch
on**. A **`ComputeBackend` (deployment profile)** decides *where* the core runs (a local process
vs a Cloud Run GPU job); a **`StorageBackend`** decides *where bytes come from* (local FS / USB
/ mounted Google Drive / bucket). **One startup config** picks both. We ship **truly-local**
(local GPU + in-process stitch, zero cloud) and **cloud-serving** (upload `<2GB` / gdrive /
bucket ? a Cloud Run GPU job runs detect **and** stitch, so stitch's compute is metered), plus
**cpu-mock** for CI. The single most important caveat: detection is **byte-identical only on the
same GPU** (cross-GPU is sub-pixel-different, per the 2026-06-20 deploy report) ? so the
**parity guarantee is enforced at the WINDOW + STITCH layer**, not detector-CSV bytes.
16
17    ## 1. Problem & goal
18    - **Why now:** DECOUPLED_COMPUTE proved compute *portability* (M0?M2 + M3.5 on a GCP L4)
and is archived; its deferred **M3 (serving endpoint + per-video billing) + M4 (scale-to-zero +
cost guard)** become this project. The owner needs **both** a hosted service *and* a truly-local
mode, with the **core (detect+stitch) unaffected** by which one runs.
19    - **The two user-facing outcomes** (owner, 2026-06-21):
20    - **a) Hosted service** ? operate on small **local uploads (`<2GB`)** *and* load from
**gdrive / a bucket**, spinning up **Cloud Run for both rally segmentation AND stitching**
(stitching is compute-intensive ? run it where its cost is accounted for).
21    - **b) Truly-local** ? on a local machine with videos on **local drives/USB + a local
GPU**, run inference **and** stitching **completely locally**.
22    - **"Good" looks like:** the same `video ? rallies ? reel` core gives identical windows
+ stitched ranges whether it runs on a laptop or Cloud Run; switching is a **config/startup
choice**, never a code branch in the core.
23
24    ## 2. Decisions locked
25
26    | # | Decision | Source | Implication |
27    |---|-----|-----|-----|
28    | D1 | The core is a **pure 3-stage function** (DETECT/WINDOW/STITCH); **no profile
branch inside it**. | design `[verified]` | All profile difference lives in config + the two
registries. |
29    | D2 | **Two registries:** `ComputeBackend` (where) selected by `deployment.profile` +
`compute_target`; `StorageBackend` (what-bytes) by `input_backend`/`storage.backend`. | ADR-014
| Realizes PLATFORM_ARCHITECTURE's `local`|`hosted`|`byo_worker`. |
30    | D3 | **Profiles shipped:** `truly-local`, `cloud-serving`, `cpu-mock` (CI);
`byo_worker` reserved/deferred. | ADR-014 | Enum/seam reserved so BYO isn't precluded. |
31    | D4 | In **cloud-serving, STITCH runs on Cloud Run** (co-located with detect) so
CPU/wall + egress are metered into the per-job cost ledger. | owner (a) | Stitch-on-laptop would
hide real cost. |
32    | D5 | **Parity enforced at WINDOW+STITCH** (byte-exact for a fixed trajectory);
**detect** is byte-exact only same-GPU. | deploy report 2026-06-20 | Cross-GPU asserted at the
rally/window level, never CSV bytes. |
33    | D6 | **Default-OFF, smallest-safe-first, one PR per milestone**; any core-caller
change (e.g. configurable output base) ships behind a flag + a Launch Report. | [[launch-report-
convention]] | Zero-regression discipline. |
34    | D7 | **Cloud Run cold-start: SCALE-TO-ZERO** ? accept the ~1?3 min cold-start (it's an
async job: upload?poll?reel, amortized over the multi-minute job). **No warm pool** (that

```

~\$500/mo idle would break per-video billing). | owner 2026-06-21 | M6/M7. Revisit a warm lane only on a UX complaint. |

35 | D8 | ****Pricebook = a versioned DB table****; cost computed in ****USD**** (matches GCP billing = one source of truth), ****displayed in INR**** at a configurable FX rate (Bangalore market); re-versioned when GCP prices change. | owner 2026-06-21 | M8. |

36 | D9 | ****Edge auth = Cloudflare Access**** (reuse the site's existing CF). Lock the Cloud Run ****ingress to the CF proxy**** + ****verify the `Cf-Access` JWT signature**** (so a direct hit to the Cloud Run URL can't spoof the identity header). | owner 2026-06-21 | M9. One identity system. |

37 | D10 | ****Free-tier "data-for-reels" trade:**** free reels in exchange for ****training rights**** (uploaded footage + derived trajectories/labels); ****paid tier = no-train**** (privacy is the paid feature). ****Carve-outs:**** train ONLY on attested ****ADULT**** footage (minor footage ? reel yes, training pool ****NO**** ? DPDP/GDPR need verifiable parental consent); train on ****DERIVED**** data + ****auto-delete raw****; ****ToS counsel-reviewed****; live training-on-uploads ****gated to M9**** (public signup). Truly-local needs no DPA. | owner 2026-06-21 | M9 + D12; ties [[paper-coauthor-aim]] / PERSONALIZATION flywheel. |

38 | D11 | ****Quality floor: Gemini handoff ON**** for cloud-serving until the owned model clears the #172 ship-gate (e.g. ?0.70 multi-court) ? then flip via a ****Launch Report****; the owned model runs in ****shadow/eval**** meanwhile. (Gemini = serving/inference ONLY, never a training source ? CLAUDE.md guardrail.) | owner 2026-06-21 | M7 + D12. |

39 | D12 | ****Data-flywheel harness is IN SCOPE** (owner add, Q5): ****capture the (consented, adult) uploaded video + the Gemini reel/rally output ? **reliably store**** in the per-video workspace/bucket ? ****run shadow evals**** (owned-vs-Gemini, dual-eval-set) ? ****promote good ones to the golden TRAINING set**** (human-reviewed labels) so the owned model climbs toward the D11 floor (then Gemini flips off). | owner 2026-06-21 | new ****M11****; ties D10 (consent) + `data\_pipeline/` + the eval/experiment harness. |

40 | D13 | ****Profile is user-switchable ANYTIME ? a headline FEATURE.**** The same user runs ****local today, cloud tomorrow, back to local**** ? it's just a config/startup choice; the pure core gives ****equivalent**** output either way. ****Advertise it**** ("your machine *or* our cloud ? your call, switch anytime"). | owner 2026-06-21 | Honest caveat: equivalent at the ****rally/window**** level, not bit-identical across GPUs (D5); a single job runs in one profile, switching is ****between* jobs.** |

41 | D14 | ****NORTHSTAR ? operate toward this:**** a ****mobile, app-like**** flow ? point at a ****Google Drive**** video ? ****fully cloud-native**** run ? the stitched, ****ready-to-share**** reel lands ****back in Drive, next to the source****. | owner 2026-06-21 | Implies a ****`gdrive-api` (OAuth) StorageBackend**** (read source + write reel back) ? distinct from the local ****mount**** backend; ****resolves \$8 #7****, new ****M12****. The mobile UI is the khelsutra track; the engine must support it (async jobs + status + Drive round-trip / signed URL). Net-new scope (OAuth + Drive write); not necessarily v1, but the design must not preclude it (the ****StorageBackend` ABC accommodates it.**) |

42 | D15 | ****Auto-detect SUGGESTS, the user CONFIRMS ? execution is NEVER a surprise.**** A GPU owner may still choose cloud; ****cloud (= spend) is never entered silently****; the active profile is always explicit + surfaced. | owner 2026-06-21 | Amends \$4.3 (auto-detect = a proposed default, not an auto-decision). |

43 | D16 | ****M6/M7 implementation SHAPE + the cheap \$8 picks (owner 2026-06-21):**** (a) ****Cloud Run JOBS**** (not Services-with-GPU) for the detect+stitch unit, ****bucket-poll for completion**** (reuses `JobWaiting`/`claim\_due\_job` verbatim, fits async upload?poll?reel + scale-to-zero D7); (b) ****stitch ENCODE = libx264**** (protects the D5 byte-determinism/parity guarantee + the L4 parity test) ? ****NVDEC for DECODE only**** (a speed lever that never touches output bytes); ****NVENC encode is DEFERRED behind a config knob****, revisit only if M7 cost data justifies relaxing parity (\$8 Q8); (c) ****input cap = hard-reject `<2GB`**** for v1 (a config cap so raising to 4GB later is one line; matches the shipped M2 chunked-upload threshold ? \$8 Q9). | owner 2026-06-21 | M6/M7. Resolves \$8 Q2 (co-located, = D4), Q8, Q9, Q10 (dir name confirmed). |

44 | D17 | ****Real cloud verification BUDGET = \$100 / ?10k cap (owner 2026-06-21):**** the owner authorized actual GCP spend for due-diligence + production-grade testing/verification of M5/M7 (real L4 run, the M5 runlog, the M7 DEPLOY_REPORT), ****provided the cap is respected**** ? confirm exact cmd + hourly cost before each spend, prefer the smallest viable run + cheapest-adequate GPU, and ****DELETE every resource after**** ([[gcp-vm-always-delete]]). Removes the M6/M7 "spend" gate within the envelope; the calibrated pricebook (M8) + counsel ToS (M9) + OAuth app (M12) stay owner/counsel-side. | owner 2026-06-21 | Unblocks the real M5/M7 runs (not just the code) within the cap. |

45

46 ## 3. Foundation ? evolution / ADR lineage ? ****trace the idea end-to-end****

47 This project is the latest step in a long decision chain; each ADR contributed a piece and a ****subtlety that carries forward****:

```

48
49 | ADR / doc | Contributed | Subtlety carried into this project |
50 |---|---|---|
51 | **ADR-005** | Permissive licensing (MIT/Apache-2.0/BSD only) | The engine + any served
model + the **container image** (ffmpeg, fonts, weights) stay commercial-clean. |
52 | **ADR-008** | Owned-model topology; **train==serve featurizer single-sourcing** | The
"core unaffected across profiles" is the same parity discipline, extended from train/serve to
**local/cloud**. |
53 | **ADR-009** | KhelSutra Guru; **local-first delivery** | The **truly-local profile**
is the direct continuation ? privacy/offline self-host stays first-class. |
54 | **ADR-010** | DECOUPLED: DO?GCP pivot | The cloud-compute origin (GPU + co-located
bucket). |
55 | **ADR-011** | DECOUPLED scope = M0?M2+M3.5; **deferred M3 (serving) + M4
(billing/cost-guard)**; override the "rent nothing / not a service" posture | **This project
realizes M3/M4.** Carries $06's owner-gated gates: **DPA/consent for non-owner uploads,
collector `md5` keying, WASB-C3**. |
56 | **ADR-012** | GPU-native; TPU deferred; **Cloud Run+GPU scale-to-zero = serving
model**; **NVDEC** named as the decode lever | The serving substrate + the 7-step seed scope
this expands. |
57 | **ADR-013** | Drop DO; **GCP sole compute stack**; $200 DO credits ? non-compute only
| The provider for cloud-serving; DO Spaces remains only a possible S3 *storage* target. |
58 | **? ADR-014 (new)** | **Compute-decoupled serving:** one pure core, profile-selected
compute+storage; stitch-where-metered | This doc. **Refines** ADR-011/012, does **not**
supersede them. |
59 | PLATFORM_ARCHITECTURE.md | The `ComputeBackend`/`StorageBackend` registries | The
abstraction this realizes in code. |
60 | VIDEO_LOCALITY_MODEL.md / DESKTOP_APP_PLAN.md | L0 fully-local tier / `LocalDesktop`
backend | The truly-local profile = these, productionized. |
61 | 07-COMPUTE-ABSTRACTION (archived) | `DeviceContext`/`DeviceProfile` (designed) | WASB
hardcodes `device='cuda'` with **no CPU fallback** ? a portability gap M4 closes. |
62 | DEPLOY_REPORT_GCP_2026-06-20 (archived) | First real L4 run; **cross-GPU sub-pixel
parity finding** | Why D5 (parity at window level, not CSV bytes). |
63
64 ## 4. Design / architecture
65 See [ `architecture.svg` ](architecture.svg) for the picture. **The core never changes;
the box around it does.**
66
67 ### 4.1 The core contract (must stay byte/behaviour-identical) `[verified]`
68 1. **DETECT (GPU):** `DetectorRunner.run_predict(video_path, output_dir, video_id) ?
CSV[Frame,Visibility,X,Y]` (`backend/pipeline/detectors/base.py:164`). Three interchangeable
impls ? `WasbRunner` (WSL), `NativeWasbRunner` (Linux GPU), `StubDetectorRunner` (CPU mock) ?
emit the **identical CSV schema**. The trajectory CSV is the stable artifact boundary.
69 2. **WINDOW (CPU pure fn):** `trajectory_to_action_windows(points, fps, frame_width, ?)
? windows` (`tracknet_runner.py:281`). A shared `@staticmethod`, zero GPU/IO/randomness ? same
trajectory + config = byte-identical candidate windows. Used by every runner *and* the
eval/regression stack verbatim.
70 3. **STITCH (CPU/ffmpeg):** `VideoCompiler.compile_highlights(raw_video, segments,
out_name) ? reel` (`stitcher/compiler.py:303`). Pure FS+subprocess (merge ? ffmpeg ? watermark
? per-clip libx264 trim ? concat). Deterministic for a given input + ranges + watermark config.
71
72 **Non-goal (the trap to avoid):** *no* code branch inside detect/window/stitch keyed on
profile ? the same discipline the experiment harness already enforces (a disabled cue is byte-
identical to CONTROL).
73
74 ### 4.2 The deployment profiles
75 | Profile | Compute | Storage | Orchestration | When |
76 |---|---|---|---|---|
77 | **truly-local** | `compute_target=local_gpu`; `detector_impl=native` (Linux) / `auto`
(WSL); stitch **in-process** | `local` (or `gdrive` = mounted Drive) | `python -m backend.main`
or `worker infer` CLI; existing async job worker | Self-host / offline / privacy / dev-on-a-GPU-
box ? owner (b) |
78 | **cloud-serving** | `compute_target=cloud_run`; `detector_impl=native` (L4, cuda);
**detect AND stitch on the same Cloud Run job** | `gcs` (or gdrive mount / assembled upload);
per-video workspace; reels via signed URL | Cloud Run control plane enqueues ? Cloud Run GPU job
pulls/runs/pushes; `WAIT_AND_RESUME` reschedules the control-plane job; scale-to-zero | Hosted

```

```

SaaS ? owner (a) |
79      | **cpu-mock** | `compute_target=cpu_mock`; `detector_impl=stub` (synth/replay) |
`local`/in-memory fakes | `run_all_tests.py` / pytest | CI, regression, profile-parity, GPU-free
dev |
80      | *byo_worker* (deferred) | tenant GPU via outbound pull agent | tenant's own bucket
(signed URLs) | long-poll pull; same job table scoped by tenant | Privacy-sensitive enterprise ?
**DEFERRED**, enum/seam reserved |
81
82      ### 4.3 Profile selection (the "startup/setup config") `[design]`
83      ONE new top-level `deployment` section on `AppConfig` + preset application + env auto-
detect in `load_config`:
84      - `deployment.profile` ? {truly-local | cloud-serving | cpu-mock}` (empty = today's
manual knobs, **fully backward-compatible**).
85      - Selecting a profile applies a **coherent preset bundle** of *existing* knobs that
today drift independently: **INPUT** (new `input_backend`/`input_root`, parallel to the output-
side `StorageConfig`), **COMPUTE** (new `compute_target` enum locking `detector_impl` +
`skip_ai_handoff` + workspace strategy), **STORAGE**
(`storage_backend`/`workspace_root`/`single_folder_per_video`, all already exist).
86      - **Precedence:** built-in defaults ? profile preset ? `config.json` ? env
(`RALLY_DETECTOR_IMPL`, `WASB_*`, `DEPLOYMENT_PROFILE`, ?) ? worker `--set k=v`. Every knob
stays individually overridable.
87      - **Env auto-detect SUGGESTS, the user CONFIRMS (D15 ? no surprise execution):** the env
gives a *proposed* default (`K_SERVICE` ? cloud-serving; `CI=true` ? cpu-mock; else
`torch.cuda.is_available()` ? truly-local), but the user **explicitly confirms or overrides** it
? a GPU owner may still pick cloud, and **cloud (= spend) is NEVER entered silently**. The
active profile is always surfaced (logged/shown). **Per-profile startup checks fail/warn fast**
(cloud-serving without GCS creds/GPU ? loud error before any job).
88      - The worker (`cmd_infer`/`cmd_train`) is **already profile-agnostic** (reads
`config.storage`, accepts `--config`/`--set`) ? no worker rewrite.
89
90      ### 4.4 Compute placement ? and *why stitch runs on Cloud Run* `[design]`
91      DETECT on GPU everywhere; WINDOW CPU-pure everywhere; STITCH is real CPU work. In
**cloud-serving, stitch is co-located in the Cloud Run job** so `gpu_active_seconds` (detect) +
`wall_seconds` (detect+stitch) + `bytes_out` (reel egress) land on **one metered invocation** ?
the per-job cost ledger. Running stitch on a laptop would leave that cost
**invisible/unattributed** ? exactly what the owner wants to avoid. (Stitch stays synchronous in
the GPU job for the common one-reel case; a queued CPU-only stitch fallback only if compiles get
bursty ? the job table + `claim_due_job` already supports both.)
92
93      ### 4.5 Reuse map ? most of the seams already exist `[verified]`
94      **Exists:** the 3-stage core; `_resolve_runner` (the single compute seam);
`StubDetectorRunner` (`replay_csv` = byte-exact $0 anchor); `StorageBackend` ABC + 4 backends +
`StorageRef.parse_uri`; the worker CLI (fetch/put, `--config`/`--set`); the async job control
plane (`jobs` table, `claim_due_job` CAS); `ExecutionPolicy WAIT_AND_RESUME`/`JobWaiting`
(carries to Cloud Run dispatch verbatim); the chunked-upload router (<2GB); `skip_ai_handoff`
(LocalNoAI); storage fakes; golden fixtures + experiment harness. **Partial:** per-video
workspace helper (default-OFF); `DeviceContext` (designed); edge-auth (v4 `user_id` columns
exist). **Net-new:** the `deployment` section; `input_backend`; configurable output base (kill
hardcoded `output`); Cloud Run dispatch shim + container image; cost ledger + pricebook; edge-
auth header extraction; startup env detection.
95
96      ### 4.6 The data-flywheel harness (D12 / M11) `[design]`
97      Gemini-on (D11) ships good reels now; this harness is **how we earn the right to flip
Gemini off**. On a **consented, adult** cloud-serving job (the D10 trade), the pipeline
additionally:
98      1. **Captures** the source upload + the Gemini reel + the rally output
(intervals/report) into the per-video workspace (`workspaces/<video_id>/`), instead of the no-
consent auto-delete path.
99      2. **Reliably stores** them durably in the bucket (the consented-retention bucket; raw
video kept only for the training pool, not the no-consent path).
100     3. **Runs shadow evals** ? the owned model scores the same video alongside Gemini
(`backend/eval/experiment.compare` + the dual-eval-set, [[eval-dual-set-regression]]) ? tracks
the owned-vs-Gemini gap per job (the live read on "is the floor reachable yet?").
101     4. **Promotes to golden** ? captured videos that pass review become **golden TRAINING
rows** (human-reviewed labels via the annotation flow ? `data_pipeline/`), growing the corpus ?

```

the owned model climbs to the D11 floor ? flip Gemini off via a Launch Report.

102 This is the consent?capture?eval?goldenize **flywheel** (the cloud realization of the PERSONALIZATION contribution model): every consented free reel makes the owned model better. Reuses `data\_pipeline/`, `backend/eval/`, and the M9 consent gate; **adults-only**, raw-retention gated by the D10 consent.

103

104 ## 6. Honest limits ? what this does NOT do / prove

105 - A green CI suite proves **plumbing + determinism (window+stitch) + profile-parity** for \$0 ? it does **NOT** prove field quality, nor that real-GPU detection is good (that's owner-side, real-video).

106 - **Cross-GPU detection is NOT byte-identical** ? parity is a window-level claim, not a CSV-byte claim.

107 - The cloud `0.1.0-l4` weights are **n=2 plumbing-grade** ? cloud-serving ships with **Gemini handoff ON** until the owned model clears a floor (open Q).

108 - This does not resolve **WASB-C3** (sharing a trained model) or the **DPA/consent** gate ? both remain owner/counsel-gated before public, non-owner serving.

109

110 ## 7. Deliverables & progress tracker ? **source of truth**

111 Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per row**, off `origin/master`, no stacking. Default-OFF where it touches core callers.

112

ID	Deliverable	Depends	Gated?	Status	PR
M0	This design doc + project dir + `runlogs/` + ADR-014			owner sign-off	
M1	`deployment.profile` + `compute_target` config + preset bundles + precedence/auto-detect; unit tests (no behaviour change, profile='' default)			safe	
M2	`input_backend`/`input_root`; wire main CLI/API input through `StorageRef.parse_uri` (local=identity); storage-fake tests			safe	
M3	Configurable output/scratch base ? kill hardcoded `output/` (`trajectory_hybrid:237,313`, `yolo_hybrid:407`, `gemini`); **DEFAULT stays `output/`**; golden + stub-E2E byte-identical			default-OFF + Launch Report on flip	
M4	`DeviceContext` + WASB **CPU-fallback**; unified healthcheck/device wiring			safe (cuda default unchanged)	
M5	**truly-local profile** end-to-end (preset + startup checks); first committed **runlog** (truly-local sample run)			owner real-GPU	
M6	Cloud Run GPU **dispatch shim** (control plane ? job via storage ? re-claim) + **containerized worker image** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests			owner (GCP spend)	
M7	**cloud-serving profile** e2e on L4: upload`<2GB` + gdrive/bucket ? detect+stitch on Cloud Run ? reel via signed URL; **DEPLOY_REPORT** (posterity, sample runs + contact sheet)			owner (spend)	
M8	Per-job **cost ledger** (v6 migration: `owner_id, gpu_active_seconds, wall_seconds, bytes_out, cost_usd, cost_breakdown_json`) + **pricebook** + aggregation + `api/.../cost`			owner (billing)	
M9	**Edge auth** (Cf-Access extraction) + per-tenant scoping on `user_id`; DPA/consent gate wiring			owner (privacy/counsel)	
M10	`byo_worker` / zero-infra-BYO ? **design-only, DEFERRED**			owner approval	
M11	**Data-flywheel harness (D12)**: on a consented adult job, **capture** the upload + Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval** owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the golden TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model climbs to the D11 floor. Reuses `data_pipeline/` + `backend/eval/` + the consent gate (M9).			owner (consent/privacy)	
M12	**gdrive-api` (OAuth) StorageBackend (D14 Northstar)**: cloud reads the				

```

source from + writes the stitched reel **back to the user's Google Drive next to the source**
(per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local
mount backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? |
128
129     **Testing strategy is a first-class deliverable** ?
[ `TESTING_STRATEGY.md` ](TESTING_STRATEGY.md). **Run logs / sample runs for posterity** ?
[ `runlogs/` ](runlogs/).
130
131     ## 8. Open questions ? owner *(blocking-gates vs nice-to-knows)*
132     **? The 5 gating questions are LOCKED (owner, 2026-06-21) ? see §2 D7?D12:**
133     1. ? **Cold-start ? D7** (scale-to-zero, accept cold-start).
134     3. ? **Pricebook ? D8** (versioned DB table; USD internal / INR display).
135     4. ? **Edge auth ? D9** (Cloudflare Access; verify Cf-Access JWT, lock ingress to
proxy).
136     5. ? **DPA/consent ? D10** (free-tier data-for-reels trade; adults-only; derived-data;
counsel-gated to M9).
137     6. ? **Quality floor ? D11** + the **data-flywheel harness D12 / M11** (Gemini-ON until
the owned model clears the floor; capture?store?eval?goldenize meanwhile).
138
139     **? The remaining smaller picks are LOCKED (owner, 2026-06-21) ? see §2 D16:**
140     2. ? **Stitch placement ? co-located** in the detect job (= D4; one metered unit; the
job table supports a future CPU-split non-breaking, revisit only on bursty-compile metrics).
141     7. ? **RESOLVED ? D14:** cloud needs a **`gdrive-api` (OAuth) StorageBackend** (read the
source from + write the reel back to the user's Drive) ? the Northstar's Drive round-trip ?
distinct from the local **mount** backend (which stays for truly-local). New milestone **M12**.
142     8. ? **Codec ? D16:** **libx264 ENCODE** (protect parity) + **NVDEC DECODE-only**
(speed, no output-byte change); NVENC-encode deferred behind a config knob.
143     9. ? **Input cap ? D16:** **hard-reject <2GB** for v1, as a config cap (raise to 4GB
later in one line).
144     10. ? **Directory name ? `COMPUTE_DECOUPLED_SERVING/` CONFIRMED final** (already the
name everywhere + in ADR-014); `CLOUD_RUN_GPU_SERVING/` dropped.
145
146     ## 9. Definition of done
147     - [ ] M1?M4 land (config + input + output-base + device layer), each green on golden
cross-OS + experiment no-regression.
148     - [ ] **Profile-parity test green** ? truly-local vs cloud-mock produce byte-identical
windows + segment ranges.
149     - [ ] **truly-local** (M5) and **cloud-serving** (M7) each have a committed **runlog /
DEPLOY_REPORT** with a real sample run.
150     - [ ] Cost ledger (M8) makes GPU + **stitch** + egress cost visible per job; edge auth
(M9) gates multi-tenant.
151     - [ ] **Data-flywheel harness (M11)** captures?stores?shadow-evals?goldenizes consented
adult jobs; the owned model is on a measured path to the D11 floor.
152     - [ ] ? flip Status DONE, archive to `docs/archives/past_projects/`.
153
154     ## 10. References
155     **ADR lineage:** ADR-005/008/009/010/011/012/013 ? **ADR-014**
(`docs/archives/decisions/DECISIONS.md`). **Strategy:** `PLATFORM_ARCHITECTURE.md`,
`HOSTING_PLAN.md`, `VIDEO_LOCALITY_MODEL.md`, `DESKTOP_APP_PLAN.md`,
`LOCAL_APPLIANCE_ARCHITECTURE.md` (khelsutra). **Archived predecessor:**
`docs/archives/past_projects/DECOUPLED_COMPUTE/` (esp. `05-COST-ATTRIBUTION.md`, `07-COMPUTE-
ABSTRACTION.md`, `06-RISKS-AND-OPEN-QUESTIONS.md`, `DEPLOY_REPORT_GCP_2026-06-20.md`). **Code
anchors `[verified]`:** `backend/pipeline/detectors/base.py:164`, `tracknet_runner.py:281`,
`stitcher/compiler.py:303`, `segmenters/trajectory_hybrid.py:65,237`, `backend/storage/*`,
`backend/worker/`__main__.py:88, `backend/api/{uploads,job_worker}.py`,
`backend/infrastructure/database.py`. **Design provenance:** workflow `wf_56274ff0` (4 code-
mappers ? synthesis; 58 findings).
156
157     ---
158     ### Changelog
159     - 2026-06-21 ? created from the design workflow output; M0 in progress.
160     - 2026-06-21 ? **M0 APPROVED** ; **M1 landed**
([#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279)):
`deployment.profile` + `compute_target` + preset bundles + precedence/auto-detect-as-suggestion
(D15) on `AppConfig`. Pure/additive, `profile=''` default == today's behaviour (zero core

```

change); presets touch only existing knobs (no M2 `input\_backend` / M3 output-base over-reach). Suite 1068 green, cov 84.11%.

161 - 2026-06-21 ? ****M2 landed**** ([#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280)): `input\_backend`/`input\_root` + URI-aware main API/CLI input via `resolve\_input\_uri`/`acquire\_local\_input` (local = identity passthrough, byte-for-byte unchanged; bucket/Drive fetched to scratch). Unsupported scheme ? clean 400; `local://` accepted on the resolved key; hosted-mode rejects non-local URIs; source URI kept for DB/report provenance. Suite 1105 green, cov 84.84%.

162 - 2026-06-21 ? ****M3 landed**** ([#282](https://github.com/Khelsutra/badminton-highlight-indexer/pull/282)): configurable output/scratch base via `backend/utils/paths.output\_base(config)` ? the 3 segmenters' chunk/handoff/temp dirs root under `output.output\_dir` (default `"output"`, byte-identical; default-OFF ? no Launch Report). Fixes the latent bug where scratch ignored `--output-dir`. Suite 1114 green, cov ~84.9%.

163 - 2026-06-21 ? ****M4 landed**** ([#283](https://github.com/Khelsutra/badminton-highlight-indexer/pull/283)): `DeviceContext` + native WASB ****CPU-fallback**** via a new `device="auto"` (cuda-if-available-else-cpu); explicit `"cuda"` stays strict (no silent slow-CPU downgrade), default unchanged. `"auto"` resolves before argv (never reaches `wasb\_infer.py`); CUDA probe cached. Device-selection tested offline; real CPU-inference run owner/GPU-verified. Suite 1126 green, cov 84.96%. ****? M1?M4 (the config/input/output/device layer) COMPLETE.****

164 - 2026-06-21 ? ****M5+ batch authorized + design picks locked**** (owner): the remaining \$8 picks resolved ? ****D16**** (M6 = Cloud Run ****Jobs**** + bucket-poll; ****libx264 encode + NVDEC decode-only****; ****hard-reject <2GB**** input cap; dir name confirmed) and ****D17**** (real cloud verification budget = ****\$100/?10k cap****, delete-after). Owner gave a blanket go to write+merge the ****default-OFF code**** for every remaining gated milestone (M5,M6,M8,M9-auth,M9-consent-cols,M11-plumbing,M12), one isolated PR each, reserving the gate for the real run/spend/calibration/counsel. Grounded by gate-map workflow `wf\_4849400f`.

165 - 2026-06-21 ? ****M8 code landed**** ([#288](https://github.com/Khelsutra/badminton-highlight-indexer/pull/288)): per-job ****cost ledger**** ? `backend/billing.py` (pure `usage × pricebook` math, USD internal / INR display) + a ****v6 migration**** (NULLABLE cost columns on `jobs` + a versioned `pricebook` table seeded `placeholder-v0` at \$0; additive-NUL, idempotent) + `record\_job\_cost`/`get\_job\_cost`/`get\_video\_cost` + `GET /api/jobs/{id}/cost` + `/api/videos/{id}/cost` + `BillingConfig` (****default-OFF****) + a gated `\_maybe\_record\_job\_cost` hook. Adversarial review (`wf\_498b24af`) folded a ****major silent-GPU-under-bill**** (an unpriced `gpu\_type` is now surfaced, not invisible \$0) + the ****honest no-op**** caveat (usage emission is a follow-up ? cost 0 until wired) + breakdown reconciliation. Suite green, ruff+mypy clean. ****? M8 ? ? owner inserts real GCP prices (a new pricebook version) + the worker usage-emission is a follow-up; placeholder ships \$0 = no user-facing change. v6 is the shared bump (M9-consent cols extend it).****

166 - 2026-06-21 ? ****M6 code landed**** ([#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287)): `backend/compute`/`ComputeBackend` registry ? `CloudRunCompute` dispatches a Cloud Run ****Job**** (injectable client) then ****bucket-polls**** a done-marker via `execution\_policy.poll\_until` + the storage layer (D16); `get\_compute\_backend` returns a backend ONLY for `compute\_target=cloud\_run` (else `None` ? the worker's in-process path, ****default-OFF****). The dispatched container runs ****inline**** (forced `local\_gpu`+native, never re-dispatch); the worker writes a `--done-uri` completion marker on success. `deploy/cloudrun/{Dockerfile,README}` = the L4 image (CUDA+ffmpeg+fonts+`wasb` conda env; weights staged per WASB-C3) + the M7 runbook. Adversarial review (`wf\_87b22fa4`) folded a ****blocker**** (cloud `fetch` needs a dst ? reads silently returned `{}` on gcs/s3) + a poll-wait test gap + `PolicyTimeout`?clean rc 4. Suite green, ruff+mypy clean. ****? M6 ? ? the real build/push/L4 run + DEPLOY_REPORT is M7 (owner, D17 budget).****

167 - 2026-06-21 ? ****M5 code landed**** ([#286](https://github.com/Khelsutra/badminton-highlight-indexer/pull/286)): `backend/config/startup.py` ? per-profile fail/warn-fast startup checks (cloud-serving + non-cloud storage = the one fatal coherence error; GPU/creds mismatches = warnings, the runner healthcheck stays the authority), wired into the server (`\_enforce\_startup\_or\_exit` ? exit 1) + the worker (`cmd\_infer`/`cmd\_train` ? rc 2), default-OFF (probe gated behind an active profile ? the worker stays torch-free at `profile=""`). ****L4 profile-parity test**** (`tests/test\_profile\_parity.py`): same stub trajectory ? byte-identical windows + segment ranges across truly-local / cpu-mock / no-profile + worker-vs-in-process. Adversarial 4-lens review folded (`wf\_34671ac1`). Suite green, ruff+mypy clean. ****? M5 ? ? the real-GPU truly-local runlog (L5) is owner-side**** (template pre-staged in `runlogs/`).

168

32. assistant (2026-06-21T13:53:01.635Z)

Now I have a comprehensive understanding of the architecture. Let me do web research on the

deployment options. Let me load the WebSearch tool and run parallel searches.

33. user (2026-06-21T13:53:13.013Z)

Web search results for query: "Cloud Run GPU deploy container scale-to-zero pricing 2026 L4 NVIDIA jobs"

Links: [{"title":"GPU support for services | Cloud Run | Google Cloud Documentation","url":"https://docs.cloud.google.com/run/docs/configuring/services/gpu"}, {"title":"Configure GPUs for Cloud Run jobs | Google Cloud Documentation","url":"https://docs.cloud.google.com/run/docs/configuring/jobs/gpu"}, {"title":"Cloud Run pricing | Google Cloud","url":"https://cloud.google.com/run/pricing"}, {"title":"Google Cloud Run Adds Support for NVIDIA L4 GPUs, NVIDIA NIM, and Serverless AI Inference Deployments at Scale | NVIDIA Technical Blog","url":"https://developer.nvidia.com/blog/google-cloud-run-adds-support-for-nvidia-l4-gpus-nvidia-nim-and-serverless-ai-inference-deployments-at-scale/"}, {"title":"Cloud Run | Google Cloud","url":"https://cloud.google.com/run"}, {"title":"Lessons Learned: Deployment Trade-offs with Gemma4, NVIDIA L4, Cloud Run, and Antigravity CLI | by xbill | Jun, 2026 | Medium","url":"https://xbill999.medium.com/lessons-learned-deployment-trade-offs-with-gemma4-nvidia-l4-cloud-run-and-antigravity-cli-8b67ee67da50"}, {"title":"Google Cloud Run Pricing in 2025: A Comprehensive Guide","url":"https://cloudchpr.com/blog/cloud-run-pricing"}, {"title":"Cloud Run GPUs are now generally available | Google Cloud Blog","url":"https://cloud.google.com/blog/products/serverless/cloud-run-gpus-are-now-generally-available"}, {"title":"L4 Cloud Pricing: Compare 15+ Providers (2026)","url":"https://getdeploying.com/gpus/nvidia-l4"}]

Search Results Summary

I've found comprehensive information about Cloud Run GPU deployment with NVIDIA L4 GPUs, scale-to-zero capabilities, and pricing. Here are the key findings:

GPU Support and Deployment

Cloud Run services configured to use GPU can scale down to zero for cost savings when not in use. Google provides NVIDIA L4 GPUs with 24 GB of GPU memory (VRAM), and you can configure one GPU per Cloud Run instance.

For container deployment, L4 GPU requires at least 4 CPU and at least 16Gi (16 GiB) of memory.

Cloud Run Jobs with GPU

Cloud Run jobs with GPUs work well for AI workloads such as training large language models (LLMs), performing batch or offline inference on LLMs, and handling other compute-intensive tasks like video processing and graphics rendering as background jobs. For L4 GPU, enter value `nvidia-l4` when configuring job specifications.

Pricing and Billing

In Tier 1 regions, an NVIDIA L4 is around \$0.0001867 per second without zonal redundancy, which is roughly \$0.67 per hour. You are charged for GPU resources per second, and Cloud Run automatically scales your GPU instances down to zero when no requests are received, eliminating idle costs.

Performance

Cloud Run instances with an attached L4 GPU with drivers pre-installed start in approximately 5 seconds, at which point the processes running in your container can start to use the GPU.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

34. user (2026-06-21T13:53:14.580Z)

Web search results for query: "Render "deploy to render" button render.yaml blueprint one-click deploy 2026"

Links: [{"title": "Deploy to Render Button ? Render Docs", "url": "https://render.com/docs/deploy-to-render"}, {"title": "Render Blueprints (IaC) ? Render Docs", "url": "https://render.com/docs/infrastructure-as-code"}, {"title": "Blueprint YAML Reference ? Render Docs", "url": "https://render.com/docs/blueprint-spec"}, {"title": "Use One-Click Deploys for Secure Online VS Code | Render Blog", "url": "https://render.com/blog/one-click-gitpod"}, {"title": "Deploy n8n on Render ? Render Docs", "url": "https://render.com/docs/deploy-n8n"}, {"title": "Render . OpenClaw", "url": "https://docs.openclaw.ai/install/render"}, {"title": "Deploying your web app to Render | First Draft", "url": "https://firstdraft.com/lessons/114-deploying-to-render"}, {"title": "Django | Render", "url": "https://render.com/templates/django"}, {"title": "Deploying on Render ? Render Docs", "url": "https://render.com/docs/deploy"}, {"title": "Deploy to a Server on Render.com - Blitz.js", "url": "https://blitzjs.com/docs/deploy-render"}]

Based on the search results, here's information about Render's "Deploy to Render" button and render.yaml blueprint for one-click deployment:

Deploy to Render Button Overview

To use the Deploy to Render button, you create a render.yaml file describing the services you want to deploy from your repository. Once you add the Deploy to Render button to a README in your repo, your users can simply click the button, review the services to be deployed on their Render account and click approve to set everything up instantly and without any additional work.

Setting Up the Button

You add the Deploy to Render button to your GitHub/GitLab/Bitbucket README.md, or if you're not using Markdown for the README, add an image from <https://render.com/images/deploy-to-render-button.svg> linking to <https://render.com/deploy>.

render.yaml Blueprint Specification

A render.yaml is a Blueprint spec that allows you to define environment variables, secrets, disks, build directories, start commands, healthchecks, deployment regions, and other configuration settings for your application without using a GUI. Every Render Blueprint is backed by a YAML file that defines a set of interconnected services, databases, and environment groups. By default, this file has the name render.yaml and resides in your Git repository's root directory.

Auto-Deploy Considerations

When using the Deploy to Render button with a render.yaml Blueprint Spec, it is strongly advised to set `autoDeploy: false` via the render.yaml Blueprint Spec. This ensures that code pushes to the repo that contains the Deploy to Render button don't trigger an automatic deploy of every instance deployed via the Deploy to Render button.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

35. user (2026-06-21T13:53:15.345Z)

Web search results for query: "Render Railway Fly.io deploy Docker GPU support FastAPI Python 2026 comparison"

Links: [{"title": "Heroku's Dead: Railway vs Render vs Fly.io 2026 | TECHSY", "url": "https://techsy.io/en/blog/railway-vs-render-vs-fly-io"}, {"title": "Deploy a FastAPI App | Railway Guides", "url": "https://docs.railway.com/guides/fastapi"}, {"title": "FastAPI deployment options | Render", "url": "https://render.com/articles/fastapi-deployment-options"}, {"title": "Run a FastAPI app · Fly"}]

```
Docs", "url": "https://fly.io/docs/python/frameworks/fastapi/"}, {"title": "Fly.io vs Railway 2026: Faster & Cheaper? Tested - The Software Scout", "url": "https://thesoftwarecout.com/fly-io-vs-railway-2026-which-developer-platform-should-you-deploy-on/"}, {"title": "Python Hosting Options Compared: Vercel, Fly.io, Render, Railway, AWS, GCP, Azure (2025) - Nandann Creative Agency", "url": "https://www.nandann.com/blog/python-hosting-options-comparison"}, {"title": "Railway vs Render: which platform fits your workload in 2026? | Blog ? Northflank", "url": "https://northflank.com/blog/railway-vs-render"}, {"title": "Railway vs Render 2026: Pricing & Speed Tested - The Software Scout", "url": "https://thesoftwarecout.com/railway-vs-render-2026-best-platform-for-deploying-apps/"}, {"title": "7 Free Docker Hosting Platforms Tested (March 2026)", "url": "https://snapdeploy.dev/blog/free-docker-hosting-2026-platforms-compared"}]
```

Based on the search results, here's a comprehensive comparison of Render, Railway, and Fly.io for deploying Docker applications with FastAPI and Python in 2026:

Deployment & Docker Support

Fly.io offers more control and global reach but requires Docker knowledge. Fly.io requires flyctl and, in practice, a Dockerfile. Community buildpacks exist, but most Fly.io users end up writing their own Dockerfile for control.

Railway auto-detects your runtime with Railpack (the successor to Nixpacks, which is now in maintenance mode). No Dockerfile, no config file, no build commands. Railway also supports Dockerfiles but shines more with its auto-detection for common frameworks.

Render's native buildpacks handle the rest when you connect GitHub and pick your branch.

FastAPI & Python Support

Railway's guides cover how to deploy a FastAPI app on Railway in four ways: One-click deploy from a template, from a GitHub repository, using the CLI, and using a Dockerfile.

Deploying a FastAPI app on Fly.io is fast - you can be up and running in less than a minute.

Render's infrastructure properly supports ASGI servers like Uvicorn, which handle FastAPI's async event loop.

GPU Support

Fly.io now offers managed Postgres, GPU instances (A100s and L40S), Kubernetes support, object storage, and scale-to-zero capabilities.

Render does not support GPU workloads.

For GPU requirements, **Fly.io is your only option among these three platforms**.

High-Level Summary

Railway gives you usage-based simplicity, Render gives you managed production infrastructure, and Fly.io gives you global edge deployment with full Docker control.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

36. assistant (2026-06-21T13:53:18.923Z)

Now let me research the Cloudflare Access JWT verification details, "one artifact serves local+cloud" patterns, and the BYO-cloud / data privacy angles.

37. user (2026-06-21T13:53:27.285Z)

Web search results for query: "Cloudflare Access JWT verification Cf-Access-Jwt-Assertion python

aud team domain validate signature"

Links: [{"title": "Validate JWTs · Cloudflare One docs", "url": "https://developers.cloudflare.com/cloudflare-one/access-controls/applications/http-apps/authorization-cookie/validating-json/"}, {"title": "JWT Verification for Cloudflare Access - Access - Cloudflare Community", "url": "https://community.cloudflare.com/t/jwt-verification-for-cloudflare-access/67645"}, {"title": "JSON Web Tokens validation · Cloudflare API Shield docs", "url": "https://developers.cloudflare.com/api-shield/security/jwt-validation/"}, {"title": "Cloudflare Zero Trust Site Protection", "url": "https://www.aem.live/developer/cloudflare-zero-trust"}, {"title": "Validating JSON Web Tokens · Cloudflare Access docs", "url": "https://developers.cloudflare.com/access/setting-up-access/validate-jwt-tokens"}, {"title": "Application token · Cloudflare One docs", "url": "https://developers.cloudflare.com/cloudflare-one/access-controls/applications/http-apps/authorization-cookie/application-token/"}, {"title": "GitHub - delebru/CloudflareJwtValidator.NET: Middleware for simple JWT (JSON Web Token) validation on web apps behind Cloudflare Zero Trust.", "url": "https://github.com/delebru/CloudflareJwtValidator.NET"}, {"title": "Protecting APIs with JWT Validation", "url": "https://blog.cloudflare.com/protecting-apis-with-jwt-validation/"}, {"title": "Jotter: Cloudflare Access and JWTs", "url": "https://amedia.github.io/jotter/2024/cloudflare-access-and-jwts/"}, {"title": "Validate the Access token with FastAPI · Cloudflare One docs", "url": "https://developers.cloudflare.com/cloudflare-one/tutorials/fastapi/"}]

I've completed a web search on Cloudflare Access JWT verification. Here are the key findings:

Overview

When Cloudflare sends a request to your origin, the request will include an application token as a Cf-Access-Jwt-Assertion request header. The Cf-Access-Jwt-Assertion header is recommended for validation instead of the CF_Authorization cookie, since the cookie is not guaranteed to be passed.

Key Components for Verification

****Team Domain and AUD Tag:****

You will need to insert your own team domain and Application Audience (AUD) tag into the sample code. Cloudflare Access assigns a unique AUD tag to each application. The aud claim in the token payload specifies which application the JWT is valid for.

****Public Key Location:****

The public key for the signing key pair is located at <https://<your-team-name>.cloudflareaccess.com/cdn-cgi/access/certs>, where <your-team-name> is your Cloudflare One team name. By default, Access rotates the signing key every 6 weeks. This means you will need to programmatically or manually update your keys as they rotate.

Validation Process

Cloudflare signs the token with a key pair unique to your account. You should validate the token with your public key to ensure that the request came from Access and not a malicious third party.

Python Implementation

For Python implementations, you can use the PyJWT library to decode tokens with the audience claim validation, looping through available public keys until a valid signature is found.

The search results include references to multiple programming languages (Python, Go, JavaScript) with implementation examples for JWT validation in Cloudflare Access.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

38. user (2026-06-21T13:53:27.431Z)

Web search results for query: "Cloud Run ingress internal-and-cloud-load-balancing restrict Cloudflare tunnel only spoof identity header"

Links: [{"title": "Restrict network endpoint ingress for Cloud Run services | Google Cloud Documentation", "url": "https://docs.cloud.google.com/run/docs/securing/ingress"}, {"title": "Google Cloud internal load balancers support Cloud Run services | Google Cloud Blog", "url": "https://cloud.google.com/blog/products/networking/google-cloud-internal-load-balancers-support-cloud-run-services"}, {"title": "How to Configure Cloud Run Ingress Settings to Allow Only Internal Traffic from", "url": "https://oneuptime.com/blog/post/2026-02-17-how-to-configure-cloud-run-ingress-settings-to-allow-only-internal-traffic-from-your-vpc/view"}, {"title": "Securing Cloud Run Ingress Traffic | Google Codelabs", "url": "https://codelabs.developers.google.com/cloud-run-ingress-deployment"}, {"title": "Cloud Run ingress setting - Serverless Applications - Google Developer forums", "url": "https://discuss.google.dev/t/cloud-run-ingress-setting/49523"}, {"title": "Go Serverless with Google Cloud Run: A Comprehensive Guide to Deployment and Ingress | by Mokadi Surya Prasad | Medium", "url": "https://medium.com/@mokadisuryaprasad/go-serverless-with-google-cloud-run-a-comprehensive-guide-to-deployment-and-ingress-2e865d45d4d5"}, {"title": "Set up a regional internal Application Load Balancer with Cloud Run | Cloud Load Balancing | Google Cloud Documentation", "url": "https://docs.cloud.google.com/load-balancing/docs/l7-internal/setting-up-l7-internal-serverless"}, {"title": "Cloudflared Ingress Configuration | OpenCharts", "url": "https://community-charts.github.io/docs/charts/cloudflared/ingress-configuration"}, {"title": "How to configure a Cloud Run service to access an internal Cloud Run service using direct VPC egress | Google Codelabs", "url": "https://codelabs.developers.google.com/codelabs/how-to-configure-cloud-run-service-direct-vpc-egress"}]

Based on the search results, here's what I found regarding Cloud Run ingress settings with the `internal-and-cloud-load-balancing` option:

Cloud Run Ingress Settings

For services behind a load balancer, use `internal-and-cloud-load-balancing` to allow the load balancer through while still blocking direct access.

Internal and Cloud Load Balancing Configuration

When calling from Cloud Run or App Engine to a Cloud Run service that's set to "Internal" or "Internal and Cloud Load Balancing", traffic must route through a VPC network that's considered internal.

By using the restrictive ingress setting and VPC service control perimeters, you can make your Cloud Run service accessible only via the internal load balancer and resources within the secure VPC perimeter.

Security Layers

The search results emphasize a multi-layered security approach: Ingress: Controls WHERE traffic can come from (network level) IAM: Controls WHO can invoke the service (identity level)

However, the search results do not contain specific information about:

- Restricting to only Cloudflare tunnel traffic with the `internal-and-cloud-load-balancing` setting
- Spoofing identity headers
- Specific configurations to allow only Cloudflare tunnel access while blocking other sources

For detailed information about restricting Cloud Run to only Cloudflare tunnel traffic and preventing identity header spoofing, you may need to consult Google Cloud's advanced networking documentation or contact Google Cloud support directly.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

39. user (2026-06-21T13:53:31.619Z)

Web search results for query: "same Docker image local and cloud config-driven twelve-factor environment variable deployment profile pattern"

Links: [{"title": "Understanding the Twelve-Factor App Methodology", "url": "https://oneuptime.com/blog/post/2026-02-20-twelve-factor-app-guide/view"}, {"title": "Deploy 12 Factor - Compliant Apps with ConfigMaps in K8s", "url": "https://www.linkedin.com/pulse/deploy-12-factor-compliant-apps-configmaps-k8s-mbong-ekwoge"}, {"title": "Twelve-Factor Applications: How Do You Validate Your Configuration?", "url": "https://marmelab.com/blog/2018/12/05/twelve-factor-applications-how-do-you-validate-your-configuration.html"}, {"title": "Developing Twelve-Factor Apps using Amazon ECS and AWS Fargate | Amazon Web Services", "url": "https://aws.amazon.com/blogs/containers/developing-twelve-factor-apps-using-amazon-ecs-and-aws-fargate/"}, {"title": "12 factor app configuration vs leaking environment variables · GitHub", "url": "https://gist.github.com/telent/9742059"}, {"title": "The 12-Factor App: A Blueprint for Modern .NET Cloud ...", "url": "https://runninghill.co.za/blog/the-12-factor-app-a-blueprint-for-modern-net-cloud-native-applications"}, {"title": "The Twelve-Factor App Methodology", "url": "https://www.pyryne.com/post/the-twelve-factor-app-methodology"}, {"title": "Modern Twelve-Factor Apps with Docker", "url": "https://www.convex.com/blog/modern-twelve-factor-apps-with-docker"}, {"title": "How to Build 12 Factor Microservices on Docker - Part 2", "url": "https://hub.packtpub.com/how-to-build-12-factor-design-microservices-on-docker-part-2/"}]

I've performed the web search for your query. Here are the key findings related to using the same Docker image for local and cloud deployments with config-driven, twelve-factor environment variable patterns:

Key Principles

Docker promotes the concept of "build once, deploy many times," where the same version of the Docker image can be used in both non-production and production environments.

The twelve-factor approach involves using the same image with the same config and same environment but with different commands for various tasks like database migrations.

Configuration Management

You write code to look for data in an environment variable?locally, you set it to localhost; in the cloud, you set it to refer to the Kubernetes Service that exposes the database component to your cluster.

The twelve-factor app stores config in environment variables. The twelve-factor methodology prescribes strict separation of configuration from code, with configurations that vary among deployment environments not stored with application code?the code for your application must be the same across your deployment environments.

Release-Stage Pattern

The pipeline combines your deployment artifact with any environment specific configuration to form a release. A Dockerfile defines how to create an immutable Docker image (the build), and running a container from this image with specific environment variables (config) constitutes a release that is then run.

Backing Services

A twelve-factor app makes no distinction between local and third-party services?swapping a local PostgreSQL for Amazon RDS should require only a config change.

This pattern ensures portability and consistency across development, staging, and production environments.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

40. assistant (2026-06-21T13:53:35.715Z)

Let me fetch the key authoritative pages for precise, citable details: the Cloud Run FastAPI Cloudflare Access tutorial, the Render deploy-to-render button doc, and Fly.io GPU pricing.

41. user (2026-06-21T13:53:42.450Z)

Validating Cloudflare Access JWT in FastAPI

Key Components

Header/Location: The JWT is transmitted via the `CF_Authorization` cookie, not a traditional header.

Library: PyJWT is used for token decoding and validation.

Public Key Retrieval: Keys are fetched from Cloudflare's certificate endpoint using: `"https://<your-team-name>.cloudflareaccess.com/cdn-cgi/access/certs"`. The response contains a JWK set that must be converted to RSA public keys.

Configuration Requirements:

- `POLICY_AUD`: Your application's Audience tag
- `TEAM_DOMAIN`: Your Cloudflare Access team domain

Validation Steps

1. **Extract token** from `request.cookies["CF_Authorization"]`
2. **Fetch public keys** from the certs URL and convert from JWK format
3. **Decode and verify** the JWT using PyJWT with:
 - The extracted public key
 - Algorithm: RS256
 - The configured AUD tag for audience validation
4. **Apply as dependency** using FastAPI's `Depends()` mechanism to protect routes

As stated in the documentation: "Loop through the keys since we can't pass the key set to the decoder" to find the matching key for verification.

42. user (2026-06-21T13:53:43.739Z)

Web search results for query: "Fly.io GPU pricing per second L40S A100 scale to zero machines 2026"

Links: [{"title": "Fly.io Resource Pricing · Fly Docs", "url": "https://fly.io/docs/about/pricing/"}, {"title": "Demystifying Fly.io Pricing: Pay for What You Use, Simply - Oreate AI Blog", "url": "https://www.oreateai.com/blog/demystifying-flyio-pricing-pay-for-what-you-use-simply/872f18e7ca3388c305cc5fbb5da484"}, {"title": "Pricing · Fly", "url": "https://fly.io/pricing/"}, {"title": "GPUs are generally available! - Fresh Produce - Fly.io", "url": "https://community.fly.io/t/gpus-are-generally-available/18198"}, {"title": "Fly.io | Review, Pricing & Alternatives", "url": "https://getdeploying.com/flyio"}, {"title": "We're Cutting L40S Prices In Half · The Fly Blog", "url": "https://fly.io/blog/cutting-prices-for-l40s-gpus-in-half/"}, {"title": "Fly GPUs · Fly Docs", "url": "https://fly.io/docs/gpus/"}, {"title": "Calculating GPU pricing for image generation - Questions / Help - Fly.io", "url": "https://community.fly.io/t/calculating-gpu-pricing-for-image-generation/23629"}, {"title": "Fly.io Pricing Calculator · Fly", "url": "https://fly.io/calculator?m=0_0_0_0_0&f=c&b=sin.0&a=no_none&r=shared_0_1_iad&t=10_100_5&u=0_1_100&g=1_shared_730_1_256_sin_1024_0"}, {"title": "Getting Started with Fly GPUs · Fly Docs", "url": "https://fly.io/docs/gpus/getting-started-gpus/"}]

Based on the search results, here's what I found about Fly.io GPU pricing:

Current GPU Pricing

On-demand GPU pricing includes: A10: \$0.75/hr per GPU, L40S: \$0.70/hr per GPU, A100 40G PCIe: \$1.25/hr per GPU, and A100 80G SXM: \$1.50/hr per GPU.

Billing and Scale to Zero

GPUs are billed by the second when the attached Machine is running. This means you can convert the hourly rates to per-second pricing by dividing by 3,600 seconds.

Important Deprecation Notice

However, there's a significant development: GPUs are deprecated and will be unavailable after August 1. This indicates that Fly.io's GPU offering is being phased out, so the pricing information may not be relevant for future deployments.

The search results don't contain specific information about "scale to zero machines" features or 2026-specific updates beyond the deprecation notice. For the most current information about Fly.io's GPU services and any new offerings that may have replaced the deprecated GPU service, you may want to check their official pricing page directly.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

43. user (2026-06-21T13:53:47.836Z)

Web search results for query: "data residency privacy user video upload cloud GDPR DPDP India processing consent ephemeral delete"

Links: [{"title":"Data Protection and Privacy Acts in India: A 2026 Compliance Guide","url":"https://sisainfosec.com/blogs/data-protection-and-privacy-laws-in-india-2025"}, {"title":"Data Retention and Deletion Under India's DPDP Rules","url":"https://ksandk.com/data-protection-and-data-privacy/data-retention-and-deletion-under-indias-dpdp-rules/"}, {"title":"Digital Personal Data Protection Act | Data Privacy India","url":"https://www.segrite.com/understanding-data-privacy-and-dpdp-act/"}, {"title":"Data protection laws in India - Data Protection Laws of the World","url":"https://www.dlapiperdataprotection.com/?t=law&c=IN"}, {"title":"Digital Personal Data Protection Act India: Compliance Guide 2026","url":"https://www.atlassystems.com/blog/digital-personal-data-protection-act-india"}, {"title":"Everything You Need to Know About the DPDP Act India, 2023 - Knovos","url":"https://www.knovos.com/blog/everything-you-need-to-know-about-the-dpdp-act-india/"}, {"title":"How to Implement GDPR Data Processing Compliance Controls in Google Cloud","url":"https://oneuptime.com/blog/post/2026-02-17-how-to-implement-gdpr-data-processing-compliance-controls-in-google-cloud/view"}, {"title":"India's Data Protection Laws | Zscaler Privacy","url":"https://www.zscaler.com/privacy-compliance/india-dpdp"}, {"title":"India Digital Personal Data Protection Act (DPDPA 2025): Updated Guide - CookieYes","url":"https://www.cookieyes.com/blog/india-digital-personal-data-protection-act-dpdp/"}, {"title":"125+ DPDPA FAQ | Digital Personal Data Protection Act 2023 & DPDP Rules 2025","url":"https://www.dpdpa.com/dpdp-faq.html"}]

I've completed the web search for your complex privacy and data protection query. Here's a summary of the key findings:

India's DPDP Act Overview

India's DPDP Act has entered its intensive enforcement phase in 2026, and the DPDP Act adopts a more principles-based and streamlined approach, limited to digital personal data, with strong emphasis on consent as the primary ground for processing.

Key Rights and Obligations

Citizens can exercise access rights to request personal data summaries, correction rights to

force fixes of inaccurate data, deletion rights to demand erasure once the business purpose concludes, and the ability to withdraw consent at any stage.

Data Retention and Deletion

The DPDP Rules create a detailed system for managing retention and deletion, and while global privacy discussions often focus on consent and notice, the true operational burden lies in how long data is kept, why it is retained, and how it is deleted.

Consent Requirements

Consent must be "free, specific, informed, unconditional, and unambiguous," with clear affirmative action signifying consent.

Comparison with GDPR

Key differences include consent as the primary lawful basis in DPDP versus GDPR's six lawful bases, DPDP's broader definition of children (under 18) versus GDPR's (under 16), and different penalty structures (DPDP up to ₹250 crore, GDPR up to 4% of global revenue).

Cross-Border Data Transfers

The final Act permits cross-border transfers except to countries explicitly restricted by the Indian government, meaning businesses operating globally will not face blanket restrictions but must remain vigilant for future government notifications.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

44. assistant (2026-06-21T13:53:52.035Z)

I now have enough. Let me verify two engine-specific facts: (1) the CORS `` + 127.0.0.1 bind claim in `backend/main.py`, and (2) the `get\_compute\_backend` factory wiring, to ground the "one artifact serves both" claim precisely.

45. user (2026-06-21T13:53:54.388Z)

```
20:from fastapi.middleware.cors import CORSMiddleware
46:# Enable CORS for the local web interface. allow_origins='' with allow_credentials=True is
50:    CORSMiddleware,
51:    allow_origins=[""],
735:    uvicorn.run(app, host="127.0.0.1", port=args.port)
---COMPUTE FACTORY---
4:``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``
32:def get_compute_backend(
39:    """Return the compute backend for ``config.deployment.compute_target``, or ``None`` to
run
42:    Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
43:    Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
47:    target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
48:    if target != "cloud_run":
49:        return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
52:    from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
54:    job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
```

46. assistant (2026-06-21T13:54:02.203Z)

All key claims are now grounded. I have verified the architecture, the deploy scripts, the auth/billing/profile design, and gathered current web research. Let me compile the structured

research output.

The picture is clear:

- ``get_compute_backend`` returns ``CloudRunCompute`` only for ``compute_target=cloud_run``, else ``None`` (inline) ? this is the exact "one artifact, config-selected" seam (``backend/compute/__init__.py:47-49``).
- The same ``deploy/cloudrun/Dockerfile`` worker image runs both inline-on-VM and dispatched-to-Cloud-Run-Job; the control plane just sets ``DEPLOYMENT_PROFILE``.
- CORS is ``*`` + bound to ``127.0.0.1`` (``backend/main.py:51,735``), so the edge gate (Cloudflare Access, D9) is the sole tenancy boundary ? no app auth.
- Render has no GPU; Fly GPUs are being deprecated (Aug 1); Railway has no GPU. Cloud Run GPU (L4, ~\$0.67/hr, scale-to-zero, ~5s GPU-ready cold start) is the one fit that also matches the per-video billing economics (M8) and ADR-013 (GCP-only).

47. user (2026-06-21T13:56:04.804Z)

Output does not match required schema: root: must have required property 'fit_for_this_app'

48. assistant (2026-06-21T13:56:08.918Z)

The ``fit_for_this_app`` got nested inside the parameter incorrectly due to a malformed tag. Let me resubmit with correct structure.

49. user (2026-06-21T13:57:42.480Z)

Output does not match required schema: root: must have required property 'fit_for_this_app'

50. assistant (2026-06-21T13:57:46.023Z)

The ``fit_for_this_app`` parameter is being malformed ? I have a stray ``</invoke>`` and ``</parameter>`` mid-content breaking the parse. Let me write it cleanly with the parameter properly closed.

51. user (2026-06-21T13:59:06.562Z)

Output does not match required schema: root: must have required property 'fit_for_this_app'

52. assistant (2026-06-21T13:59:10.582Z)

The XML close tag mismatch: I wrote ``</parameter>`` for the ``recommendation`` open tag but used ``name="recommendation"`` opened with ``<parameter name="recommendation">`` ? that's correct. The issue is ``fit_for_this_app`` is opened with ``<parameter name="fit_for_this_app">`` (missing the ``antml:`` prefix) but closed with ``</parameter>``. Let me fix the open tag.

53. user (2026-06-21T14:00:34.368Z)

Structured output provided successfully

54. assistant (2026-06-21T14:00:37.321Z)

Research complete. The structured output was submitted successfully.